

Universidad Austral de Chile

Facultad de Ciencias de la Ingeniería
Escuela de Ingeniería Civil en Informática

INTEGRACIÓN DE DATOS Y RESILIENCIA COMUNITARIA EN LA DIMENSIÓN SOCIOCOMUNITARIA

Proyecto para optar al título de
Ingeniero Civil en Informática

PROFESOR PATROCINANTE:
CRISTIAN OLIVARES-RODRÍGUEZ
INGENIERO CIVIL INFORMÁTICO
MASTER EN VISIÓN POR COMPUTADORA E INTELIGENCIA ARTIFICIAL
DOCTOR EN INGENIERÍA

ESTEBAN NICOLÁS TEJEDA WEBAR

VALDIVIA – CHILE
2026

DEDICATORIA

AGRADECIMIENTOS

ÍNDICE

Dedicatoria I

Agradecimientos II

Resumen VIII

Abstract IX

1. INTRODUCCIÓN 1
- 1.1. Resiliencia comunitaria . . . 1
- 1.2. Modelo CORE . . . 2
- 1.3. Descripción del proyecto . . . 2
  - 1.3.1. Estado del arte . . . 3
  - 1.3.2. Descripción de la innovación . . . 3
  - 1.3.3. Impacto del proyecto . . . 4
- 1.4. Objetivos . . . 4
  - 1.4.1. Objetivo general . . . 4
  - 1.4.2. Objetivos específicos . . . 4

2. MARCO TEÓRICO 5
- 2.1. Bases teóricas . . . 5
  - 2.1.1. Proceso ETL . . . 5
  - 2.1.2. Web scraping . . . 5
  - 2.1.3. Patrones de diseño . . . 6
- 2.2. Objetivo y pregunta de revisión . . . 6
- 2.3. Método de revisión . . . 6
- 2.4. Fuentes y estrategias de búsquedas . . . 6
- 2.5. Cadena de búsquedas . . . 7
- 2.6. Criterios de selección . . . 7
- 2.7. Extracción de la información . . . 7
- 2.8. Estudios incluidos y excluidos . . . 8
- 2.9. Información extraída general . . . 8
- 2.10. Principales hallazgos y factores claves . . . 9

3. VIABILIDAD DE INDICADORES 12
- 3.1. Population Poverty (PP) . . . 12
- 3.2. Social Capital – Civic Participation (SCCP) . . . 13
- 3.3. Social Capital – Civic Organizations (SCCO) . . . 13
- 3.4. Social Capital – Emergency Organizations (SCEO) . . . 13
- 3.5. Tsunami Drills . . . 14
- 3.6. Special Needs Population (SNP) . . . 14

3.7. Conclusiones del análisis de viabilidad 15

4. HERRAMIENTAS DE DESARROLLO 16
4.1. Lenguaje de programación 17
4.2. Base de datos 17
4.3. Entorno de ejecución 18
4.3.1. Librerías principales 18
4.3.2. Librerías secundarias 19
4.4. Contenedores 19
4.5. Visualización de datos 20
4.6. Sistema de control de versiones 20

5. DISEÑO DE PROTOTIPO 21
5.1. Alcance del prototipo 21
5.2. Fuentes de datos exploradas 21
5.3. Modelo de datos inicial 23
5.4. Clases de extracción 25
5.5. Aprendizajes y transición a la arquitectura final 25

6. DESARROLLO E IMPLEMENTACIÓN 27
6.1. Arquitectura general 27
6.2. Patrones de diseño 28
6.2.1. Patrón Builder 28
6.2.2. Patrón Adapter 29
6.2.3. Patrón Factory y Singleton 30
6.2.4. Clase abstracta ScrapeBase 30
6.3. Tipos de conexión implementados 31
6.4. Módulos implementados 32
6.4.1. Módulo emergencias-desastres 32
6.4.2. Módulo biblioteca-congreso-nacional 33
6.5. Normalización de unidades territoriales 34
6.6. Manejo de errores 35
6.7. Automatización 36
6.8. API REST 36
6.9. Testing 38
6.10. Volatilidad de las fuentes web 38

7. RESULTADOS 40
7.1. Datos extraídos 40
7.1.1. Tsunami Drills 40
7.1.2. Resultados calculados de Tsunami Drills 41
7.1.3. Tasa de Pobreza por Ingresos 43
7.1.4. Organizaciones Comunitarias 44
7.2. La API REST en funcionamiento 45

7.3. Visualización en Metabase 46
7.4. Validación de la extracción 47
7.5. Evitación de duplicados 48
7.6. Rendimiento de la extracción 49
7.7. Ejecución automatizada 50

8. CONCLUSIONES **51**
8.1. Conclusiones generales 51
8.2. Conclusiones por objetivo 51
8.2.1. Objetivo Específico 1: dimensión institucional 52
8.2.2. Objetivo Específico 2: dimensión sociocomunitaria 52
8.2.3. Objetivo Específico 3: API y analíticas 53
8.3. Aportes de la arquitectura 53
8.4. Sobre la calidad de las fuentes de datos públicas 54
8.5. Limitaciones 55
8.6. Trabajo futuro 56

REFERENCIAS **59**

A. Indicadores del modelo CORE **60**

B. Ejemplos de datos extraídos **61**
B.1. Colección emergencia-desastres 61
B.2. Colección tasa-pobreza-ingresos 61
B.3. Colección organizaciones-comunitarias 62
B.4. Colección indicator-results 62

C. Declaración de un indicador **63**
C.1. Declaración mediante el IndicatorBuilder 63
C.2. Extracción de los datos desde el HTML 63

ÍNDICE DE TABLAS

1. Resumen de la información extraída por artículo 9

2. Resumen de la investigación sobre Population Poverty 12
3. Resumen de la investigación sobre Civic Participation 13
4. Resumen de la investigación sobre Civic Organizations 13
5. Resumen de la investigación sobre Tsunami Drills 14
6. Resumen de la investigación sobre Special Needs Population 15

7. Stack tecnológico y versiones utilizadas 16

8. Limitaciones del prototipo y su resolución en la arquitectura final 26

9. Tipos de conexión implementados en los módulos 31
10. *Adapters* disponibles para futuros módulos 32
11. Grafías distintas de una misma región en la fuente del MINEDUC 35
12. *Endpoints* de la API REST 37

13. Registros extraídos por indicador de simulacros 40
14. Resultados calculados por indicador de simulacros 41
15. Datos extraídos de tasa de pobreza por ingresos 43
16. Comparación entre los datos de referencia y los datos extraídos 48
17. Tiempos de ejecución por etapa del flujo ETL, en milisegundos 49

18. Cumplimiento de los objetivos específicos 52

ÍNDICE DE FIGURAS

1. Número de artículos por año 8
2. Cantidad de veces que se analizó cada dimensión en las investigaciones . 10
3. Cantidad de indicadores totales por dimensión 10
4. Promedio de indicadores por aparición de dimensión en los artículos . . . 11

5. Información sociocomunitaria sobre tasa de pobreza en el año 2017 . . . 22
6. Información sociocomunitaria sobre población carente de servicios básicos y hogares hacinados en el año 2018 23
7. *Interface* para la información de la página web 24
8. *Interface* para tasa de pobreza 24
9. *Interface* para población carente de servicios básicos y hogares hacinados 24
10. Lista de clases utilizadas para la extracción de la información 25

11. Arquitectura general del sistema: las tres capas y su relación con las fuentes externas y la base de datos 27
12. Configuración del indicador `simulacros-2021` mediante el *IndicatorBuilder* 28
13. Diagrama de clases de los *adapters* con sus métodos abstractos 29
14. Diagrama de secuencia del flujo ETL completo 31
15. Registro almacenado por el módulo emergencias-desastres 33
16. Registros almacenados por cada sub-indicador del módulo BCN: tasa de pobreza (arriba) y organizaciones comunitarias (abajo) 34
17. Definición de un *endpoint* en la colección Bruno OpenCollection 37

18. Extracto de los registros de simulacros-2021 mostrando fecha, tipo y región 41
19. Resultado calculado de simulacros-2021 consultado desde la API 42
20. Gráfico de barras de simulacros por año en Metabase 42
21. Distribución acumulada de simulacros por región (2021-2023) en Metabase 43
22. Respuesta de la API para el indicador de tasa de pobreza por ingresos . . 44
23. Gráfico comparativo de pobreza CASEN 2017 frente a CASEN 2022 en Metabase 44
24. Evolución temporal de las organizaciones comunitarias de Valdivia en Metabase 45
25. Catálogo de módulos e indicadores devuelto por el *endpoint* raíz de la API 46
26. *Dashboard* de Metabase con los indicadores del modelo CORE 47

27. Indicadores de la dimensión social del modelo CORE 60

RESUMEN

La presente tesis aborda la sistematización del proceso de captura de indicadores cuantitativos sociocomunitarios e institucionales mediante la extracción automatizada de datos de la web, con el objetivo de alimentar el modelo CORE de resiliencia comunitaria ante tsunamis en zonas costeras de Chile.

Se desarrolló un sistema de extracción, transformación y carga (ETL) basado en una arquitectura modular de *adapters* intercambiables, implementado en TypeScript con Node.js y MongoDB. El sistema permite agregar nuevos indicadores y fuentes de datos sin modificar la lógica central, soportando distintos tipos de conexión y estrategias de extracción: *web scraping* de HTML, extracción de tablas y consumo de APIs JSON.

Se implementaron cinco indicadores distribuidos en dos módulos: simulacros de emergencia (dimensión institucional) y tasa de pobreza junto con organizaciones comunitarias (dimensión sociocomunitaria). El sistema extrajo un total de 29 registros de simulacros, datos de pobreza a nivel comunal, regional y nacional, y 17 registros históricos de organizaciones comunitarias. Los datos fueron validados contra extracciones manuales y se verificó el correcto funcionamiento de los mecanismos de evitación de duplicados.

Los resultados se ponen a disposición mediante una API REST y *dashboards* en Metabase, permitiendo al equipo de investigación consultar los indicadores de forma directa sin procesamiento manual.

Palabras clave: resiliencia comunitaria, ETL, *web scraping*, modelo CORE, indicadores sociocomunitarios, TypeScript.

ABSTRACT

This thesis addresses the systematization of the process of capturing quantitative socio-community and institutional indicators through automated web data extraction, aiming to feed the CORE community resilience model for tsunami-prone coastal areas in Chile.

An extraction, transformation, and loading (ETL) system was developed based on a modular architecture of interchangeable adapters, implemented in TypeScript with Node.js and MongoDB. The system allows adding new indicators and data sources without modifying the core logic, supporting different connection types and extraction strategies: HTML web scraping, table extraction, and JSON API consumption.

Five indicators were implemented across two modules: emergency drills (institutional dimension) and poverty rate together with community organizations (socio-community dimension). The system extracted a total of 29 drill records, poverty data at municipal, regional, and national levels, and 17 historical records of community organizations. The data was validated against manual extractions and the correct functioning of duplicate avoidance mechanisms was verified.

Results are made available through a REST API and Metabase dashboards, allowing the research team to query indicators directly without manual processing.

Keywords: community resilience, ETL, web scraping, CORE model, socio-community indicators, TypeScript.

1. INTRODUCCIÓN

La ubicación geográfica de Chile, situado principalmente entre las placas tectónicas de Nazca y Sudamérica, determina una alta actividad sísmica en el país. A esto se suma la presencia de fallas geológicas, actividad volcánica y efectos de geología local, los cuales pueden provocar desde incendios y erupciones, hasta numerosos sismos y tsunamis.

Uno de los desastres naturales más recordados sigue siendo el "27F": el terremoto de magnitud 8,8 ocurrido el 27 de febrero de 2010, cuyo epicentro se localizó frente a la costa sur de la Región del Maule y que generó un tsunami que impactó las costas chilenas, destruyendo localidades y arrasando con poblados enteros. El balance oficial registró 521 personas fallecidas y 56 desaparecidas (Memoria Chilena, Biblioteca Nacional de Chile, s.f.).

Con lo anterior como antecedente, se debe agregar que no es posible predecir un desastre natural como un terremoto. Aun así, si bien no se puede saber cuándo ocurrirá, sí se puede tener una idea de cómo actuará la población afectada y cómo esta puede sobreponerse a situaciones de adversidad de esta índole.

1.1. Resiliencia comunitaria

Una de las maneras de afrontar esta problemática es utilizando un modelo de resiliencia comunitaria. Si bien no existe una definición específica que logre englobar todo lo que este representa, se utiliza la siguiente definición: "La resiliencia comunitaria se refiere por lo tanto a la capacidad del sistema social y de las instituciones para hacer frente a las adversidades y para reorganizarse posteriormente de modo que mejoren sus funciones, su estructura y su identidad" (Maguire & Cartwright, 2008).

Las diversas investigaciones en el área de la resiliencia comunitaria han dado origen a los llamados "Modelos de resiliencia comunitaria". Estos modelos varían según el área geográfica, el tipo de comunidad, el comportamiento de las poblaciones y el enfoque que se quiera dar a la investigación.

Cada modelo de resiliencia comunitaria se basa en el cálculo de distintas dimensiones, y a su vez, en cada dimensión se encuentra una diversa variedad de indicadores. Estos indicadores permiten medir los cambios de la resiliencia comunitaria y varían según el modelo que se esté aplicando.

Por lo tanto, algunos estudios afirman que el uso de un modelo de resiliencia comunitaria

puede mejorar la preparación, respuesta y recuperación ante desastres a nivel comunitario en un corto plazo, y entregar una mejor adaptación a largo plazo (Cutter et al., 2014). Si bien existen muchos modelos que atienden a determinadas poblaciones específicas, el modelo que se utiliza en este proyecto es el modelo CORE.

1.2. Modelo CORE

El modelo CORE es un modelo de resiliencia comunitaria que está centrado en las costas chilenas, utiliza dimensiones físicas, ambientales y sociales donde se encuentran 21 indicadores.

Su iniciativa nace, tal como indica Villagra et al. (2017), debido a la información sobre los indicadores que permiten a las ciudades enfrentar y adaptarse mejor a los impactos de los tsunamis, siendo esta información especialmente escasa para los países en desarrollo como Chile.

Este modelo permite evaluar el riesgo que presentan las áreas costeras susceptibles de ser afectadas por tsunamis, y con base en ello prevenir accidentes y desastres, además de mitigar las posibles consecuencias negativas con el fin de que la población se enfrente de mejor manera a dichos eventos.

Dado que el modelo CORE requiere la obtención y procesamiento de múltiples indicadores, resulta necesario contar con herramientas que automaticen este proceso. A continuación se describe el proyecto desarrollado con este fin.

1.3. Descripción del proyecto

Como se mencionó anteriormente, para el cálculo de la resiliencia comunitaria es necesario obtener la información de distintas dimensiones, las cuales a su vez requieren de diversos indicadores. Cada uno de estos indicadores posee un gran volumen de datos que debe ser descargado, transformado y calculado.

Hasta ahora, cuando se desea calcular la resiliencia comunitaria, los investigadores consumen gran parte de su tiempo en buscar manualmente dentro de diversas fuentes de internet la ubicación de esta información; ya sea en páginas web que entreguen bases de datos ya establecidas, o en la búsqueda de artículos que tengan información posiblemente relevante.

1.3.1. Estado del arte

En la literatura revisada no se identificó una solución que automatice la captura de estos indicadores. Las investigaciones de las que se tiene registro realizan el cálculo de la resiliencia comunitaria con una extracción manual de la información, que luego es ingresada y analizada con diversos programas estadísticos como SPSS y R.

En la investigación realizada por Cutter et al. (2014), el cual utiliza el modelo BRIC que está centrado en la geografía de las construcciones, se extrajeron los datos manualmente de diversas fuentes de bases de datos, la mayoría abiertas y gratuitas.

En la investigación realizada por Kamara et al. (2019), se extrajo la información para el modelo de resiliencia de manera manual e iterativa.

Por último, en la investigación realizada por Villagra et al. (2017), el cual utiliza el modelo CORE para calcular la resiliencia en las comunidades de zonas costeras; se extrajo la información descargándola en bases de datos municipales, también descargando bases de datos de la Oficina Nacional de Emergencia del Ministerio del Interior (ONEMI) y en la Corporación Nacional Forestal (CONAF), entre otros.

1.3.2. Descripción de la innovación

Se desarrolló un sistema que permite la extracción, transformación y carga de los datos. Este proceso también es conocido como ETL (*Extract, Transform and Load*) en el caso de *Business Intelligence* (BI).

Para esto, se debe comenzar con la premisa de que la información requerida se encuentra dispersa en internet de diferentes formas. Algunas páginas web permiten la extracción de esa información por medio de APIs, otras por medio descarga de archivos en formato CSV, XLSX, entre otros; además algunas páginas no brindan facilidades para la obtención de su información, por lo que es necesario utilizar técnicas de *web scraping* para obtenerla.

Una vez que la información es extraída, esta debe ser transformada. Para este proceso se debe recordar que cada cálculo varía según el indicador. Por ejemplo, en la dimensión social del modelo CORE presentada por Villagra et al. (2017), se puede encontrar un indicador llamado “Tsunami Drills” (Simulacros ante tsunamis) y se calcula con la suma de todos los simulacros realizados entre los años 2010 y 2015. Por lo tanto, los datos extraídos serían los simulacros entre estos años, pero la transformación sería la suma total de todos estos simulacros.

Finalmente, una vez que la información es transformada, esta debe ser almacenada en una

base de datos para luego ser cargada en alguna plataforma. De tal manera que el equipo de investigación pueda acceder a esta fácilmente. La plataforma permite visualizar la información por medio de distintos gráficos y en función del tiempo, junto con información extra como la web de donde fue obtenida y la fecha en la que fue extraída.

1.3.3. Impacto del proyecto

La herramienta desarrollada aporta a la investigación de resiliencia comunitaria, especialmente a la del modelo CORE, debido al ahorro de tiempo que obtiene el equipo de investigación, permitiéndole un conocimiento completo y oportuno de la resiliencia comunitaria en las regiones de Chile involucradas en el proyecto FONDECYT. Esto significa un apoyo directo a la toma de decisiones en cuanto a planificación de las ciudades, integrando la resiliencia como un criterio de análisis de las políticas públicas.

1.4. Objetivos

El presente proyecto busca responder la siguiente pregunta de investigación: ¿es posible sistematizar la captura de los indicadores cuantitativos sociocomunitarios e institucionales del modelo CORE mediante la extracción automática de datos disponibles en la web? Para abordarla se definen los objetivos que se detallan a continuación.

1.4.1. Objetivo general

Sistematizar el proceso de captura de indicadores cuantitativos sociocomunitarios e institucionales por medio de la extracción de datos de la web para alimentar un modelo de resiliencia comunitaria.

1.4.2. Objetivos específicos

1. Capturar variables para el cálculo de indicadores de la dimensión institucional desde diversas fuentes de información válidas para el modelo de resiliencia.
2. Capturar variables para el cálculo de indicadores de la dimensión sociocomunitaria desde diversas fuentes de información válidas para el modelo de resiliencia.
3. Poner a disposición los indicadores institucionales y sociocomunitarios por medio de una API y analíticas.

2. MARCO TEÓRICO

Este capítulo se organiza en dos partes. La primera presenta las bases teóricas de los conceptos técnicos sobre los que se construye el sistema desarrollado: el proceso ETL, el *web scraping* y los patrones de diseño de software. La segunda corresponde a una revisión sistemática de literatura orientada a conocer cómo los distintos modelos de resiliencia comunitaria obtienen y procesan los datos que alimentan sus indicadores.

2.1. Bases teóricas

2.1.1. Proceso ETL

El proceso de extracción, transformación y carga (ETL, por *Extract, Transform and Load*) es el conjunto de operaciones mediante el cual los datos se obtienen desde una o más fuentes de origen, se limpian y adecuan a un formato útil, y se almacenan en un repositorio de destino para su posterior análisis. Kimball y Caserta (2004) lo describen como el fundamento de todo almacén de datos (*data warehouse*), al punto de estimar que su construcción puede consumir cerca del 70 % de los recursos de un proyecto de esta naturaleza. Aunque el término proviene del ámbito del *Business Intelligence*, el proceso es aplicable a cualquier sistema que deba consolidar información dispersa y heterogénea, como ocurre con los indicadores del modelo CORE.

2.1.2. Web scraping

El *web scraping* es el conjunto de técnicas que permiten extraer de forma automática información publicada en páginas web diseñadas para ser leídas por personas y no por programas. Glez-Peña et al. (2014) lo caracterizan como la alternativa disponible cuando una fuente no ofrece una API formal de acceso a sus datos: el programa obtiene el documento HTML tal como lo recibiría un navegador y localiza en su estructura los valores de interés. Los mismos autores advierten su principal debilidad, que este proyecto experimentó de forma directa: el código de extracción depende de la estructura interna de la página, por lo que cualquier cambio en ella puede invalidar la extracción sin previo aviso.

2.1.3. Patrones de diseño

Un patrón de diseño es una solución reutilizable y documentada a un problema recurrente del diseño de software orientado a objetos, descrita con independencia del lenguaje de programación. El catálogo clásico de Gamma et al. (1994) documenta 23 patrones organizados en categorías de creación, estructura y comportamiento; entre ellos se encuentran *Builder*, *Adapter*, *Factory Method* y *Singleton*, que este proyecto emplea para estructurar su arquitectura según se detalla en el Capítulo 6.

2.2. Objetivo y pregunta de revisión

Para el desarrollo de esta aplicación se tiene la necesidad de conocer cómo los distintos modelos de resiliencia comunitaria extraen los datos para su cálculo, y a su vez, poder conocer qué tipo de cálculos realizan para obtenerlos. Por esta razón, se decidió dar respuesta a la siguiente pregunta de investigación:

¿Cómo se ha conseguido capturar los datos en los distintos modelos de resiliencia comunitaria en las dimensiones sociocomunitarias e institucionales?

2.3. Método de revisión

Para el desarrollo de esta revisión sistemática, lo primero que se hizo fue utilizar la pregunta de investigación para poder estructurar una cadena de búsqueda a aplicar. Posteriormente se seleccionan los artículos que contengan los criterios de selección dentro de los *abstract*, para luego finalmente analizar y resumir los resultados obtenidos.

2.4. Fuentes y estrategias de búsquedas

La fuente seleccionada para realizar la búsqueda es *Web of Science*. Dado que la mayoría de los artículos publicados se encuentra en inglés, se decidió construir la cadena de búsqueda en ese idioma.

Un punto importante en la búsqueda es cómo las demás investigaciones capturan o procesan los datos, por lo que se optó por *keywords* como *data processing* y *data sources*. Adicionalmente, el término “resiliencia comunitaria” aparece en inglés bajo dos formas distintas: *Community Resilience*, que puede verse como una traducción más literal, y *Di-*

saster Resilience, que en este contexto puede interpretarse como la resiliencia en caso de desastres.

2.5. Cadena de búsquedas

Con lo anterior, la cadena de búsqueda quedó definida de la siguiente manera:

(“community resilience” OR “disaster resilience”) AND (“data processing” OR “data sources”)

2.6. Criterios de selección

Una vez obtenidos los resultados de búsqueda, se procedió a seleccionar solo aquellos artículos que presenten dentro de su investigación alguna referencia en las dimensiones ya señaladas anteriormente:

- Dimensión sociocomunitaria.
- Dimensión institucional.

2.7. Extracción de la información

Se definió una pauta de extracción de información con la finalidad de poder estructurar mejor la información encontrada. Esta pauta consta de lo siguiente:

- Indicadores de dimensión sociocomunitaria.
- Indicadores de dimensión institucional.
- Método de extracción de dimensiones.
- Tecnología utilizada.
- Método de cálculo de la dimensión.

2.8. Estudios incluidos y excluidos

En primer lugar, al realizar la búsqueda con el *string* ya definido, se obtuvo un total de 47 artículos, para luego comenzar a discriminar según el contenido del *abstract*, lo cual da un total de diez artículos para poder resumir y extraer la información pertinente.

Finalmente, de los diez artículos seleccionados, solo nueve resultaron ser útiles para la investigación.

Si bien la cantidad de artículos a analizar no es alta, se puede apreciar en la Figura 1 el cómo las investigaciones de resiliencia comunitaria son contemporáneas y han ido en aumento con los años.

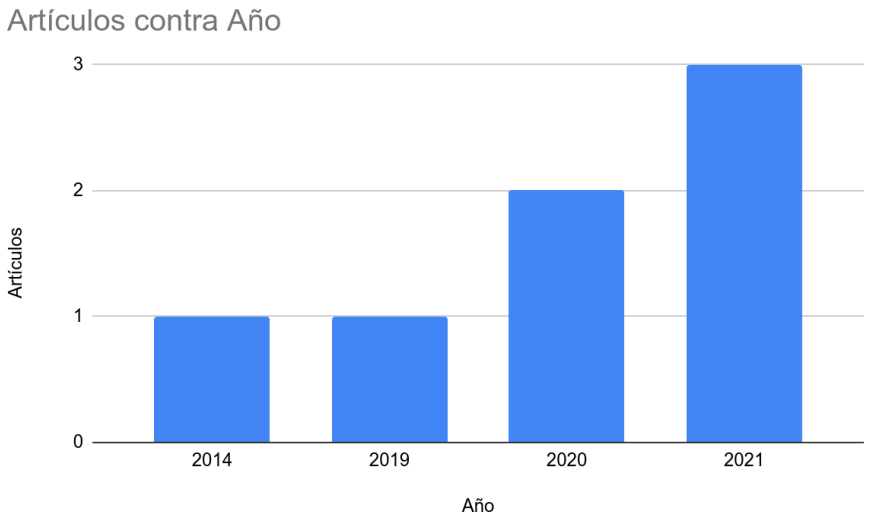


Figura 1: Número de artículos por año

2.9. Información extraída general

Toda la información que se encontró relevante en cada artículo seleccionado fue extraída y analizada para posteriormente ser incorporada como ayuda para la elaboración del proyecto de tesis (ver Tabla 1).

Tabla 1: Resumen de la información extraída por artículo

Autor(es)	**Indicadores SC**	**Indicadores Inst.**	**Método extracción**	**Tecnología**
Cutter et al. (2014)	Social Capital, Population Poverty	Emergency Services	Descarga manual de bases de datos públicas	SPSS
Villagra et al. (2017)	Civic Organizations, Population Poverty	Tsunami Drills	Descarga manual de bases de datos municipales y gubernamentales	R
Kamara et al. (2019)	Social networks, Community participation	Institutional support	Encuestas y entrevistas	SPSS
Heinzlef et al. (2020)	Social vulnerability	Emergency response	Bases de datos geoespaciales	GIS
Rumson et al. (2020)	Socioeconomic factors	Governance	Bases de datos ambientales	GIS, R
Moghadas et al. (2019)	Social capital	Institutional capacity	Encuestas	SPSS, AHP
Abrash Walton et al. (2021)	Public health outcomes	Emergency preparedness	Revisión sistemática	–
Muhamad et al. (2021)	Exposure elements	Disaster databases	Bases de datos locales	GIS
Wang et al. (2021)	Community engagement	Information dissemination	Twitter API	Python

2.10. Principales hallazgos y factores claves

Uno de los hallazgos que llaman la atención a simple vista es que si bien para este estudio se contempla la dimensión "sociocomunitaria", en todas las investigaciones analizadas estas aparecen separadas como "Dimensión social" y "Dimensión comunitaria".

Se realizó un gráfico (ver Figura 2) para apreciar de mejor manera la cantidad de dimensiones especificadas en cada investigación:

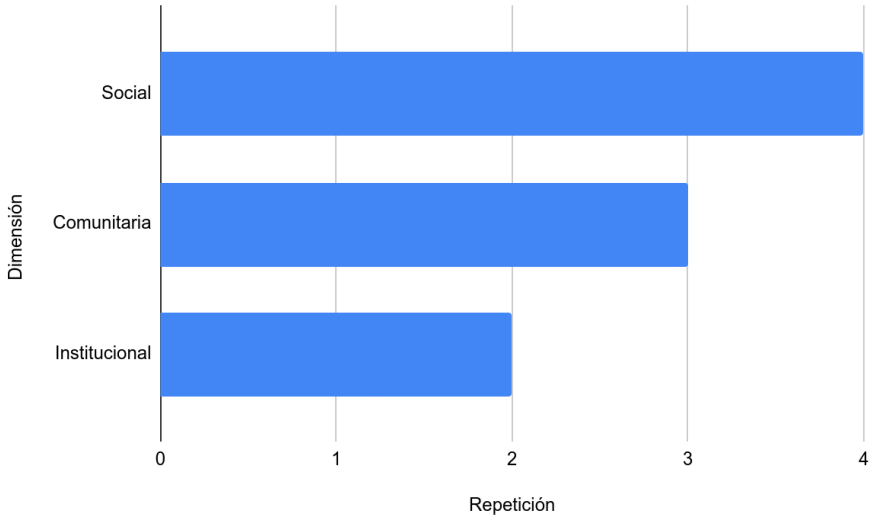


Figura 2: Cantidad de veces que se analizó cada dimensión en las investigaciones

De esta forma, se puede ver cómo la dimensión social es la más analizada con cuatro coincidencias en los *papers*, mientras que la menos analizada es la dimensión institucional, con dos coincidencias.

También se agregó la diferencia en la cantidad de indicadores por cada dimensión, como se aprecia en la Figura 3:

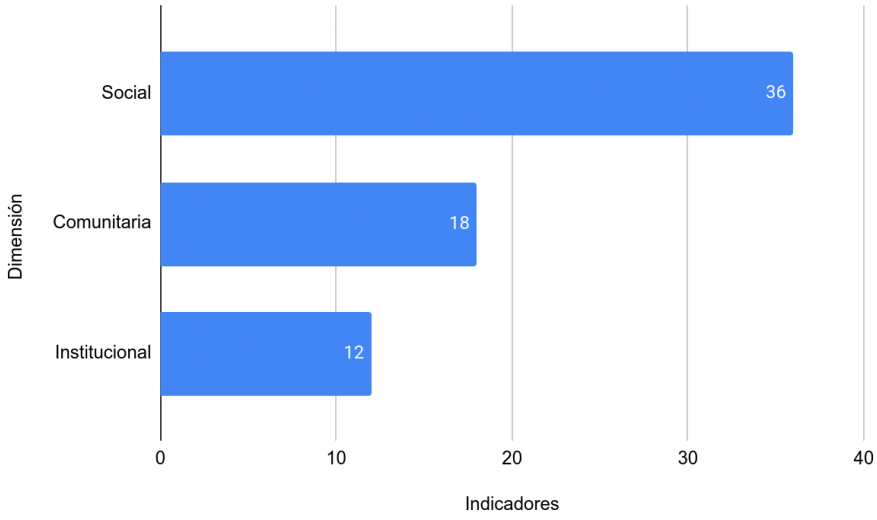


Figura 3: Cantidad de indicadores totales por dimensión

En este caso se aprecia que las dimensiones social y física son las que agrupan más indica-

dores en las investigaciones revisadas. Sin embargo, al calcular el promedio de indicadores por cada aparición de dimensión en los artículos (ver Figura 4), se observa que la dimensión institucional, a pesar de ser la menos frecuente, presenta un promedio alto de indicadores cuando aparece.

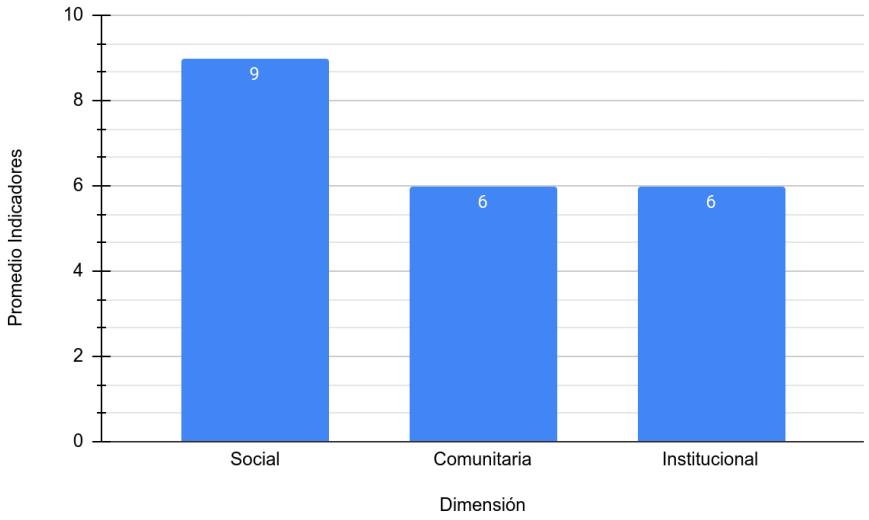


Figura 4: Promedio de indicadores por aparición de dimensión en los artículos

3. VIABILIDAD DE INDICADORES

En este capítulo se analiza la viabilidad de extracción automática de cada indicador del modelo CORE en las dimensiones sociocomunitaria e institucional. Para cada indicador se identifican las fuentes de datos disponibles, el formato en que se encuentran, la segmentación geográfica y las tareas necesarias para su incorporación al sistema.

Dentro del modelo CORE se especifican los indicadores correspondientes a las dimensiones sociocomunitaria e institucional que serán objeto de análisis (ver Anexo A). La finalidad de esta búsqueda es analizar cuáles indicadores son viables para realizar la extracción automática de sus datos.

3.1. Population Poverty (PP)

PP hace referencia a la tasa de pobreza de la población. El indicador puede expresarse como porcentaje directo o calcularse a partir de datos en bruto.

Los resultados de la investigación se pueden apreciar en la Tabla 2:

Tabla 2: Resumen de la investigación sobre Population Poverty

Fuente	**Tipo dato**	**Segm.**	**Formato**	**Notas**
BCN (Biblioteca del Congreso Nacional de Chile, s.f.-a)	Porcentaje y datos en bruto	Comunal	Tabla HTML	Requiere *web scraping*
Statista (Statista, s.f.)	Porcentaje	Nacional	Tabla	Requiere pago
DataSocial (Ministerio de Desarrollo Social, s.f.)	Datos en bruto	Regional	.xls	Descarga directa

En este caso se observa que la información de la BCN requiere técnicas de *web scraping* para obtener los datos de las tablas HTML, mientras que la fuente de Statista fue descartada por requerir una suscripción pagada. La fuente del Ministerio de Desarrollo Social ofrece la descarga directa en formato XLS.

3.2. Social Capital – Civic Participation (SCCP)

SCCP hace referencia al porcentaje de la población que participa activamente en procesos cívicos (votaciones).

Tabla 3: Resumen de la investigación sobre Civic Participation

Fuente	Tipo dato	Segm.	Formato	Notas
SERVEL (Servicio Electoral de Chile, s.f.)	Datos en bruto	Comunal	.csv	Única fuente identificada

La información es compleja de encontrar debido a que solo se identificó una fuente. El Servicio Electoral de Chile (SERVEL) provee datos en formato CSV que pueden ser descargados directamente.

3.3. Social Capital – Civic Organizations (SCCO)

SCCO hace referencia a la razón de organizaciones civiles por cada 1.000 habitantes por territorio.

Tabla 4: Resumen de la investigación sobre Civic Organizations

Fuente	Tipo dato	Segm.	Formato	Notas
BCN (Biblioteca del Congreso Nacional de Chile, s.f.-b)	Datos en bruto	Comunal	JSON, XLS, CSV, Tabla	Múltiples formatos disponibles

La BCN provee datos de organizaciones comunitarias en múltiples formatos, incluyendo una API JSON que facilita la extracción automática. Cada municipio posee sus propias fuentes de datos, pero la BCN centraliza esta información a nivel comunal.

3.4. Social Capital – Emergency Organizations (SCEO)

SCEO hace referencia a la razón de voluntarios en organizaciones de emergencia por cada 1.000 habitantes.

No se encontró una fuente de datos pública centralizada para este indicador. La información sobre voluntarios en organizaciones de emergencia depende de cada municipalidad y no se encuentra disponible de manera uniforme. Por esta razón, este indicador no pudo ser implementado en el sistema.

3.5. Tsunami Drills

Tsunami Drills hace referencia a la cantidad de simulacros de tsunami realizados en un período de tiempo definido.

Tabla 5: Resumen de la investigación sobre Tsunami Drills

Fuente	**Tipo dato**	**Segm.**	**Formato**	**Notas**
ONEMI (ONEMI, s.f.)	Datos en bruto	Nacional	Tabla HTML	Requiere *web scraping*
MINEDUC (Unidad de Riesgos y Desastres MINEDUC, 2022)	Datos en bruto	Nacional	Artículo HTML	Requiere *web scraping*

En este caso se observa que ninguno de los sitios que contienen información sobre los simulacros de tsunami presenta los datos en un formato listo para ser trabajado. La única forma de obtener esta información es por medio de técnicas de *web scraping*.

3.6. Special Needs Population (SNP)

SNP hace referencia al porcentaje de la población que padece algún tipo de discapacidad.

Tabla 6: Resumen de la investigación sobre Special Needs Population

Fuente	**Tipo dato**	**Segm.**	**Formato**	**Notas**
Obs. Social (Observatorio Social, s.f.)	Porcentaje y datos en bruto	Comunal	.xls, .sav, .dta	Descarga directa
INE (Instituto Nacional de Estadísticas, s.f.-b)	Datos en bruto	Regional	.sav, .dat	Descarga directa
BCN (Biblioteca del Congreso Nacional de Chile, s.f.-c)	Datos en bruto	Comunal	Tabla HTML	Requiere *scraping* mínimo

La mayoría de los datos se encuentra en un formato listo para ser utilizado y solo en un sitio web es necesario realizar técnicas de *web scraping*.

3.7. Conclusiones del análisis de viabilidad

Las conclusiones obtenidas en este apartado investigativo son múltiples:

En primer lugar se observa que muchos de estos datos requeridos se encuentran en un formato de fácil acceso y listos para ser descargados. En estos casos se puede diseñar el software para que tenga un fuerte énfasis en la descarga de información omitiendo el *scraping* y pasando directamente a ser transformada.

En segundo lugar, la mayoría de los datos se presenta en formato de tabla, por lo que el software podría implementar un mecanismo de detección automática de tablas.

En tercer lugar, los datos que dependen de cada municipalidad requieren una búsqueda específica por sitio web, lo que implica módulos de extracción personalizados.

En cuarto lugar, los formatos no estándar requieren módulos específicos de *scraping* adaptados a la estructura HTML de cada fuente.

4. HERRAMIENTAS DE DESARROLLO

Para la elección de las herramientas de desarrollo a utilizar se consideraron diversos parámetros, entre ellos:

- Herramientas que se ajusten a las necesidades que busca resolver el equipo de investigadores.
- La popularidad y uso de las tecnologías. De esta manera se facilita la búsqueda de desarrolladores para mantener y escalar el software a largo plazo.
- Tecnologías con las cuales ya se posee cierta experiencia.

Con la finalidad de que el entorno de desarrollo sea reproducible por el equipo de investigación, en la Tabla 7 se declaran las versiones exactas con las que fue construido y validado el sistema. Las secciones siguientes justifican cada una de estas elecciones.

Tabla 7: Stack tecnológico y versiones utilizadas

Componente	**Tecnología**	**Versión**
Lenguaje	TypeScript	5.9.3
Entorno de ejecución	Node.js	22 LTS (*jod*)
Base de datos	MongoDB	6
Framework web	Express	4.22.2
ODM	Mongoose	6.13.10
Parsing de HTML	Cheerio	1.2.0
Cliente HTTP	request-promise	4.2.6
Navegador automatizado	Puppeteer	24.40.0
Validación de esquemas	Zod	4.3.6
Logging	Pino	8.21.0
Tareas programadas	node-cron	4.2.1
Pruebas	Jest	29.7.0
Contenedores	Docker Compose	*Compose Specification*
Visualización	Metabase	`latest`
Control de versiones	Git	—

La versión de Node.js se fija mediante el archivo `.nvmrc` con el alias `lts/jod`, que corresponde a la línea 22 de soporte extendido. Esta decisión responde a que las versiones

recientes de Cheerio y de sus dependencias transitivas requieren características de módulos ECMAScript no disponibles en las líneas anteriores. La única versión no fijada es la de Metabase, cuya imagen Docker se referencia mediante la etiqueta `latest`: al tratarse de una herramienta de visualización externa al flujo ETL, su versión exacta no compromete la reproducibilidad de los resultados del sistema.

4.1. Lenguaje de programación

El lenguaje de programación elegido para el desarrollo del software es TypeScript. Un lenguaje que permite el desarrollo tanto en el *frontend* como en el *backend*.

TypeScript es un superconjunto de funcionalidades que extienden la sintaxis de Javascript, dotándolo de tipado estático y de objetos basados en clases (TypeScript, s.f.). En resumen, permite aumentar la probabilidad de detectar errores en el código, como también hacer este más legible.

La elección de este lenguaje es debido principalmente a la gran popularidad que posee Javascript, siendo en estos momentos uno de los lenguajes más utilizados a nivel mundial, con un 68,62 % de preferencia según una encuesta realizada en el año 2021 (Stack Overflow, 2021).

4.2. Base de datos

Se decidió utilizar una base de datos no relacional. Específicamente la base de datos MongoDB (MongoDB Inc., s.f.).

La elección de esta base de datos es debido a la información que se desea almacenar. Los datos deseados no poseen un esquema previamente definido, ya que cada indicador del modelo CORE posee una estructura distinta. Por lo que utilizar una base de datos relacional (como MySQL o PostgreSQL) aumentaría la complejidad en caso de querer realizar cambios en una etapa avanzada del proyecto.

Adicionalmente, MongoDB permite almacenar documentos con estructuras heterogéneas dentro de una misma colección, lo cual facilita la persistencia de datos provenientes de fuentes diversas sin necesidad de migraciones complejas.

4.3. Entorno de ejecución

El entorno de ejecución predilecto actualmente para el desarrollo de programas con Javascript/TypeScript es Node.js (OpenJS Foundation, s.f.-b).

Node.js posee una madurez de más de 15 años. Está diseñado para crear aplicaciones escalables y orientado al uso de eventos asíncronos.

Su simplicidad permite desarrollar aplicaciones en poco tiempo, posee un sistema de paquetes (npm) en constante crecimiento que permite a los desarrolladores implementar múltiples herramientas y módulos. Además, al poseer un núcleo liviano, da una ventaja en el tamaño final del software, ya que se utilizan solo las funcionalidades que se deseen implementar.

A continuación se muestra una lista de las librerías principales y secundarias más relevantes que se utilizan, junto con una pequeña descripción.

4.3.1. Librerías principales

- **cheerio** (Cheerio, s.f.): Permite analizar la estructura de las páginas web permitiendo la extracción y manipulación de los datos. Se utiliza para el *parsing* de HTML en los módulos que requieren *web scraping*.
- **express** (OpenJS Foundation, s.f.-a): Es actualmente el *framework* más popular de Node.js. Permite el manejo de peticiones HTTP y desarrollo de APIs.
- **mongoose** (Automattic, s.f.): Es un ODM que permite escribir consultas y crear esquemas para la base de datos MongoDB. Se utiliza para definir los modelos de datos de cada indicador y gestionar el almacenamiento.
- **request-promise**: Permite hacer llamadas HTTP. Se utiliza para obtener el HTML de una página web o realizar peticiones a APIs externas.[1]
- **puppeteer** (Google, s.f.): Permite la automatización de un navegador web. Se utiliza para extraer información de páginas que renderizan su contenido mediante Javascript, donde *cheerio* no es suficiente.
- **zod** (McDonnell, s.f.): Librería de validación y declaración de esquemas. Se utiliza para validar la configuración de los indicadores en tiempo de construcción, asegu-

[1] El desarrollo inicial del prototipo utilizaba la librería *request*, de interfaz basada en *callbacks*. Se migró a *request-promise*, que envuelve la misma implementación exponiendo promesas, con el fin de integrarla con la sintaxis `async/await` que emplea el flujo ETL. Ambas librerías fueron declaradas *deprecated* por sus autores en febrero de 2020 (Request, s.f.); su reemplazo se aborda en la sección 8.6 como deuda técnica.

rando que cada indicador posea los campos requeridos antes de ser registrado en el sistema.

- **pino** (Pino, s.f.): Librería de *logging* de alto rendimiento. Se utiliza como *logger* centralizado del sistema, generando registros tanto en consola (durante el desarrollo) como en archivos de log para auditoría.
- **node-cron** (Soares, s.f.): Permite la programación de tareas periódicas mediante expresiones cron. Se utiliza para automatizar la ejecución de los procesos de extracción según la frecuencia definida para cada indicador.

4.3.2. Librerías secundarias

- **eslint**: Es una herramienta que permite ayudar en la detección de errores de código y mantener este más legible.
- **prettier**: Es una herramienta de formateo de código que asegura un estilo consistente en todo el proyecto.
- **nodemon**: Es una herramienta que permite actualizar automáticamente la ejecución de la aplicación en cuanto detecta cambios en el desarrollo.
- **typescript**: Permite escribir código de TypeScript y transpilarlo a código de Javascript.
- **jest** (Jest, s.f.): *Framework* que permite realizar pruebas unitarias y de integración.

4.4. Contenedores

Para facilitar el despliegue y la reproducibilidad del entorno de desarrollo, se utiliza Docker junto con Docker Compose (Docker Inc., s.f.).

Docker permite crear contenedores aislados que encapsulan la aplicación y sus dependencias, mientras que Docker Compose permite definir y orquestar múltiples servicios de forma declarativa. En el caso de este proyecto, se definen tres servicios: la base de datos MongoDB, una interfaz de administración (Mongo Express) y una herramienta de visualización de datos (Metabase).

Esta decisión permite que cualquier integrante del equipo de investigación pueda levantar el entorno completo con un solo comando, sin necesidad de instalar ni configurar cada servicio de forma manual.

4.5. Visualización de datos

Para la visualización de los datos almacenados se utiliza Metabase (Metabase, s.f.), una herramienta de *Business Intelligence* de código abierto que permite crear *dashboards* y consultas sobre bases de datos sin necesidad de escribir código.

Metabase se conecta directamente a MongoDB y permite a los investigadores visualizar los indicadores extraídos por medio de gráficos, tablas y filtros. Los *dashboards* se actualizan automáticamente cada vez que se consultan, reflejando los datos más recientes almacenados en la base de datos.

La elección de Metabase por sobre un *frontend* personalizado (como Angular o React) se debe a que el objetivo principal del proyecto es la extracción y el procesamiento de datos, no la construcción de una interfaz gráfica. Metabase permite cumplir con el objetivo de poner a disposición los datos mediante analíticas sin requerir desarrollo adicional.

4.6. Sistema de control de versiones

Un sistema de control de versiones permite rastrear y gestionar los cambios que son realizados dentro de un código de software. Estas herramientas ayudan a los equipos de desarrollo a gestionar los cambios en el código fuente (Atlassian, s.f.).

En este caso, se optó por utilizar Git (Git, s.f.). Un sistema de control de versiones creado por Linus Torvalds, el cual está centrado en ser eficiente, confiable y mantenible. Actualmente es utilizado por más de un 93 % de los desarrolladores (Stack Overflow, 2021), convirtiéndose en una de las tecnologías indispensables para el desarrollo de software.

5. DISEÑO DE PROTOTIPO

Antes de comprometerse con una arquitectura definitiva, se construyó un prototipo cuyo propósito no era producir software de producción sino responder empíricamente una pregunta de diseño: ¿qué parte del proceso ETL varía entre una fuente de datos y otra, y qué parte permanece constante? La respuesta a esa pregunta determina qué debe ser fijo en el núcleo del sistema y qué debe ser intercambiable.

Este capítulo describe el prototipo construido, los artefactos que produjo y las limitaciones que reveló. Las decisiones arquitectónicas que se derivan de estas limitaciones se detallan en el Capítulo 6.

5.1. Alcance del prototipo

El prototipo se acotó deliberadamente a dos indicadores de la dimensión sociocomunitaria (tasa de pobreza y población carente de servicios básicos) extraídos desde una única fuente. Esta restricción fue intencional: con dos indicadores distintos sobre una misma fuente es posible observar qué código se repite entre ambos y cuál es específico de cada uno, que es precisamente la información que se buscaba. El indicador de población carente de servicios básicos no forma parte de los analizados en el Capítulo 3; se incluyó porque comparte fuente y formato de presentación con la tasa de pobreza, lo que permitía comparar dos extracciones sobre un mismo portal.

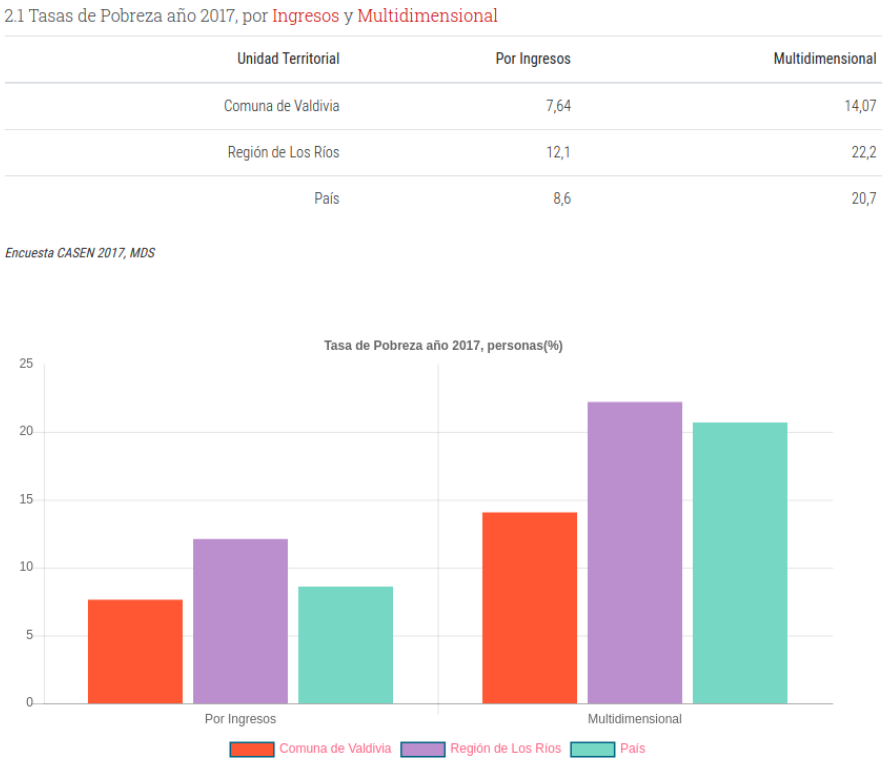
El prototipo cubrió las tres etapas del proceso ETL de extremo a extremo: obtención del HTML, extracción de los valores y persistencia en MongoDB. No se implementaron en esta etapa la programación de tareas periódicas, el manejo de errores tipados ni la exposición mediante una API, por no ser necesarios para responder la pregunta de diseño planteada.

5.2. Fuentes de datos exploradas

Se realizó una búsqueda manual en internet para encontrar páginas web que dispusieran de información pertinente a los indicadores. Si bien se encontraron diversas fuentes, la mayoría ya se encontraba en formatos descargables. En la Figura 5 y Figura 6 se muestran ejemplos de la información sociocomunitaria encontrada.

La observación relevante de esta exploración es que ambos indicadores, pese a provenir del

mismo portal y presentarse con el mismo formato visual, no compartían estructura HTML: diferían en la cantidad de columnas, en el orden de las filas y en la forma de expresar los valores. Es decir, la variabilidad entre fuentes no depende del proveedor de los datos sino de cada página en particular.



2.1 Tasas de Pobreza año 2017, por Ingresos y Multidimensional

Unidad Territorial	Por Ingresos	Multidimensional
Comuna de Valdivia	7,64	14,07
Región de Los Ríos	12,1	22,2
País	8,6	20,7

Encuesta CASEN 2017, MDS

Figura 5: Información sociocomunitaria sobre tasa de pobreza en el año 2017. Fuente: Biblioteca del Congreso Nacional de Chile (s.f.-a)

2.2 Población carente de servicios básicos y hogares hacinados (% totales, a diciembre 2018)

Unidad Territorial	Totales a Diciembre 2018 (%)	
	Personas en hogares carentes de servicios básico	Hogares hacinados
Comuna de Valdivia	10,7	13,2
Región de Los Ríos	28,4	13,8
País	14,1	15,3

Fuente: SIIS-T MDS

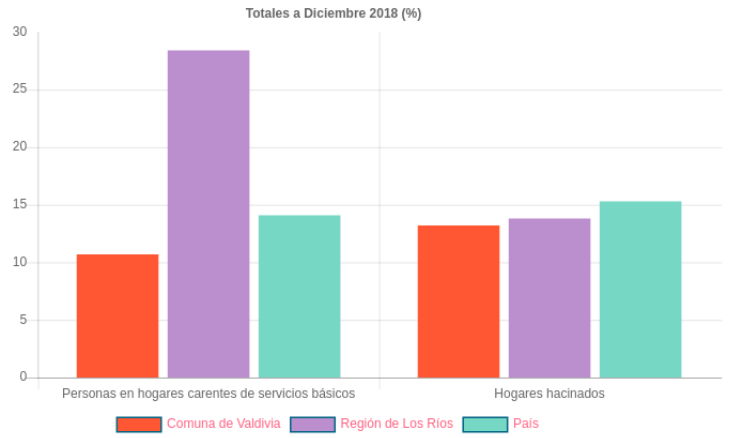


Figura 6: Información sociocomunitaria sobre población carente de servicios básicos y hogares hacinados en el año 2018. Fuente: Biblioteca del Congreso Nacional de Chile (s.f.-a)

5.3. Modelo de datos inicial

Para el modelo de la base de datos, se comenzó con la idea de almacenar la información de la página web como tal (URL de origen y fecha de extracción), para luego ir desglosando la información en modelos más específicos. En la Figura 7 se presenta la *interface* inicial.

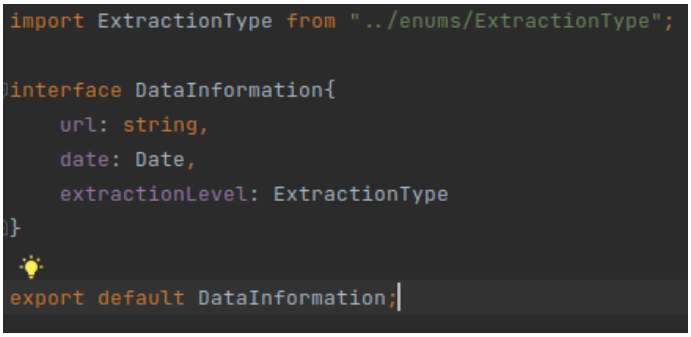


Figura 7: *Interface* para la información de la página web

Una vez obtenida la información base, se continuó con los modelos para cada indicador (ver Figuras 8 y 9). Cada indicador requirió su propia *interface*, ya que los campos de la tasa de pobreza no coinciden con los de la población carente de servicios básicos. Esta constatación, que el esquema de datos es propio de cada indicador y no puede unificarse, fue el argumento decisivo para optar por una base de datos documental por sobre una relacional, según se justificó en la sección 4.2.

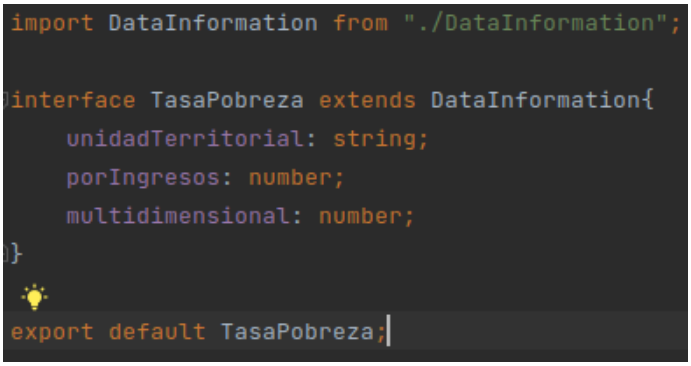


Figura 8: *Interface* para tasa de pobreza

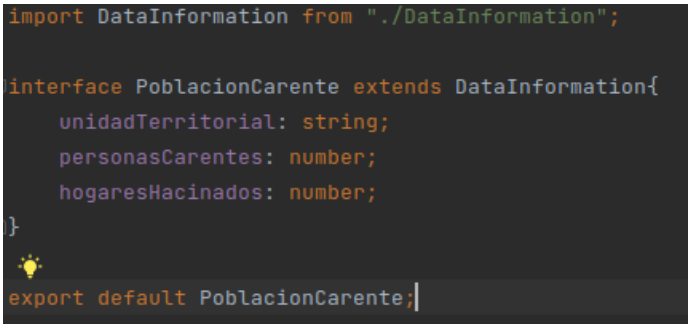


Figura 9: *Interface* para población carente de servicios básicos y hogares hacinados

5.4. Clases de extracción

Para la extracción, se crearon tres clases (ver Figura 10) encargadas de las distintas etapas del proceso.

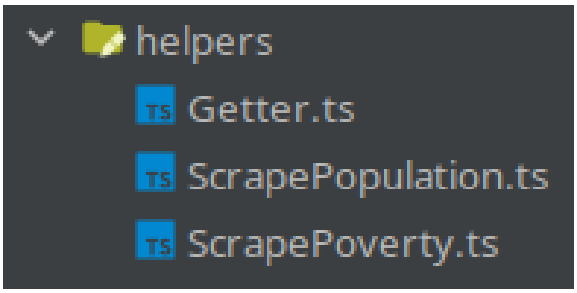


Figura 10: Lista de clases utilizadas para la extracción de la información

La organización de estas clases es en sí misma el hallazgo más importante del prototipo. La clase `Getter` concentra la obtención del contenido de una URL y es reutilizable entre indicadores. En cambio, `ScrapePoverty` y `ScrapePopulation` son una clase completa por cada indicador, y dentro de cada una conviven la extracción de los valores desde el HTML, su transformación al tipo de dato esperado y su persistencia en la base de datos.

Esta estructura presenta dos problemas que impiden escalarla a los 21 indicadores del modelo CORE. El primero es que agregar un indicador obliga a escribir una clase nueva que reimplementa las tres etapas desde cero, aunque dos de ellas sean casi idénticas a las ya existentes. El segundo es que, al estar las tres etapas mezcladas en un mismo método, no es posible probar la extracción de forma aislada ni sustituir una etapa sin reescribir las otras: un cambio en la estructura HTML de la fuente obliga a intervenir el mismo código que realiza el guardado.

5.5. Aprendizajes y transición a la arquitectura final

El prototipo permitió distinguir con evidencia qué varía entre indicadores y qué permanece constante. Lo que varía es la forma de acceder a la fuente, la manera de localizar los datos dentro del contenido obtenido, la transformación al tipo esperado y el criterio que identifica un registro como único. Lo que permanece constante es el orden en que estas etapas se ejecutan.

Esta distinción es la que define la arquitectura del sistema final: la secuencia de etapas pertenece al núcleo y se escribe una sola vez, mientras que el contenido de cada etapa se

declara por indicador. La Tabla 8 resume cómo cada limitación observada en el prototipo se traduce en un mecanismo concreto de la arquitectura final.

Tabla 8: Limitaciones del prototipo y su resolución en la arquitectura final

Limitación observada en el prototipo	**Mecanismo en la arquitectura final**
El acceso a la fuente varía (HTML estático, JSON, archivo descargable) y estaba resuelto solo para HTML	*FetchAdapter* con implementaciones concretas por tipo de conexión (sección 6.3)
Localizar los datos exige código distinto por cada página, incluso dentro de un mismo portal	*ParseAdapter* declarado por indicador (sección 6.2.2)
Los valores se obtienen como texto genérico y deben convertirse al tipo esperado	*MapperAdapter* (sección 6.2.2)
La ejecución periódica reinsertaría registros ya existentes	*HashAdapter* y restricción de unicidad en el esquema (sección 7.5)
Las tres etapas conviven en una clase por indicador, impidiendo probarlas o sustituirlas por separado	*ScrapeBase* orquesta la secuencia; cada módulo solo aporta sus *adapters* (sección 6.2.4)

En síntesis, el prototipo no se descartó ni se trasladó tal cual: su estructura de tres etapas se conservó como el flujo del sistema, pero se invirtió la relación entre lo genérico y lo específico. Donde el prototipo tenía una clase por indicador que implementaba las tres etapas, la arquitectura final tiene una implementación única de las tres etapas que recibe, por indicador, la porción que efectivamente cambia.

6. DESARROLLO E IMPLEMENTACIÓN

En este capítulo se describe la arquitectura del software desarrollado, los patrones de diseño utilizados y la implementación de los módulos de extracción de datos. La arquitectura fue diseñada con el objetivo de que agregar un nuevo indicador al sistema requiera el mínimo esfuerzo posible, manteniendo la separación de responsabilidades y la reutilización de componentes.

6.1. Arquitectura general

El sistema se estructura en tres capas principales: el *core*, los módulos y la API, tal como se muestra en la Figura 11. El *core* contiene las clases base, los *adapters* abstractos y la lógica de orquestación del flujo ETL. Los módulos son implementaciones concretas para cada fuente de datos. La API expone los indicadores al exterior mediante *endpoints* HTTP.

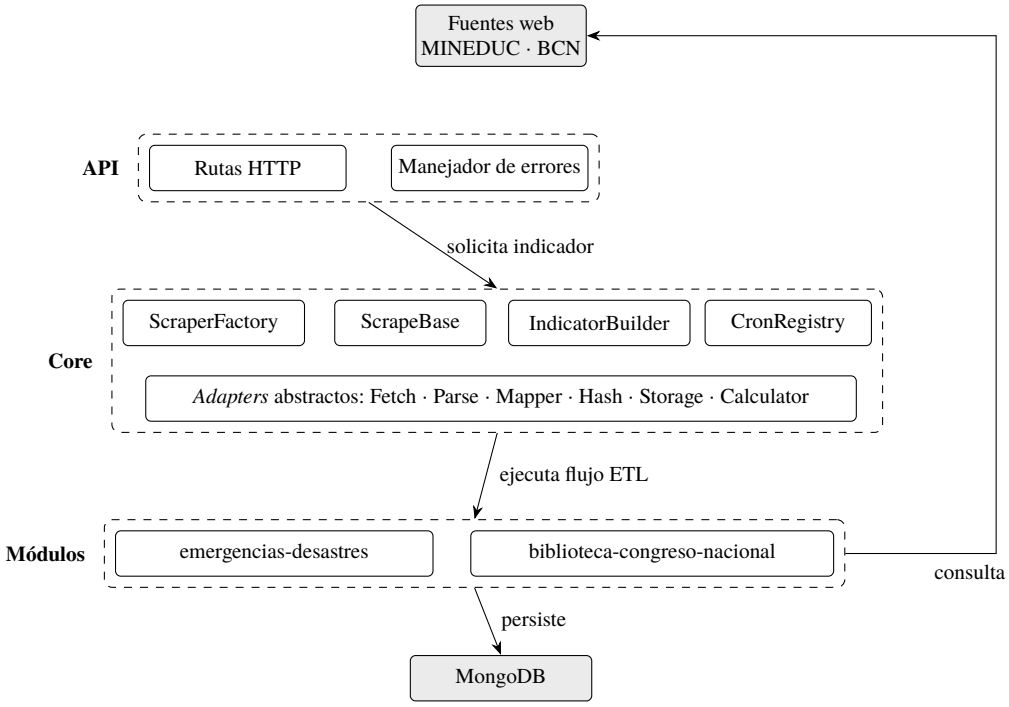


Figura 11: Arquitectura general del sistema: las tres capas y su relación con las fuentes externas y la base de datos

La comunicación entre las capas sigue un flujo unidireccional: la API recibe la solicitud,

el *ScraperFactory* obtiene el módulo correspondiente, y el módulo ejecuta el flujo ETL utilizando los *adapters* configurados para ese indicador.

6.2. Patrones de diseño

Se utilizaron diversos patrones de diseño para lograr una arquitectura extensible y con bajo acoplamiento. A continuación se describen los más relevantes.

6.2.1. Patrón Builder

El *IndicatorBuilder* permite configurar cada indicador de forma declarativa, encadenando los métodos de configuración y validando los datos con la librería Zod al momento de construir el indicador. Esto asegura que un indicador no pueda ser registrado en el sistema si le falta información obligatoria como el nombre o la URL de la fuente. La Figura 12 muestra la configuración completa de uno de los indicadores implementados.

```
"simulacros-2021": new IndicatorBuilder()
  .setName("Simulacros 2021")
  .setUrl("https://web.archive.org/web/20211026024955/https://" +
        "emergenciaydesastres.mineduc.cl/simulacros-2021/")
  .setFrequency(FREQUENCIES.once)
  .setFetchAdapter(new RequestPromiseAdapter())
  .setParseAdapter(new DateParserAdapter())
  .setMapperAdapter(new EmergenciaDesastresMapper())
  .setHashAdapter(new EmergenciaDesastresHashAdapter())
  .setStorageAdapter(new EmergenciasDesastresStorageAdapter())
  .setCalculatorAdapter(new TsunamiDrillsCalculatorAdapter())
  .build()
```

Figura 12: Configuración del indicador **`simulacros-2021`** mediante el *IndicatorBuilder*

El indicador queda descrito por completo en esta declaración: no existe código procedural asociado a **`simulacros-2021`** fuera de los *adapters* que aquí se le asignan. Agregar un indicador nuevo consiste en escribir un bloque equivalente y, cuando la fuente lo exige, implementar los *adapters* que difieran de los ya disponibles. La llamada final a **`build()`** valida la configuración con Zod, de modo que un indicador incompleto falla al iniciar la aplicación y no en el momento de ejecutar la extracción.

6.2.2. Patrón Adapter

Cada etapa del flujo ETL se define mediante un *adapter* abstracto que establece la interfaz que deben implementar las clases concretas. Esto permite que un módulo utilice un *adapter* distinto para cada etapa sin modificar la lógica central del sistema. Los *adapters* definidos son:

- **FetchAdapter**: Define cómo se obtienen los datos de la fuente (HTML, JSON, archivo).
- **ParseAdapter**: Define cómo se extrae la información estructurada de los datos crudos.
- **MapperAdapter**: Define cómo se transforman los datos extraídos al formato que se almacena en la base de datos.
- **HashAdapter**: Genera una clave única por registro para evitar duplicados.
- **StorageAdapter**: Controla el almacenamiento de los datos procesados.
- **CalculatorAdapter** (opcional): Calcula un resultado agregado a partir de los datos almacenados y lo persiste en una colección separada.

La Figura 13 presenta estos *adapters* con sus métodos abstractos, junto con las implementaciones concretas disponibles para la etapa de obtención de datos.

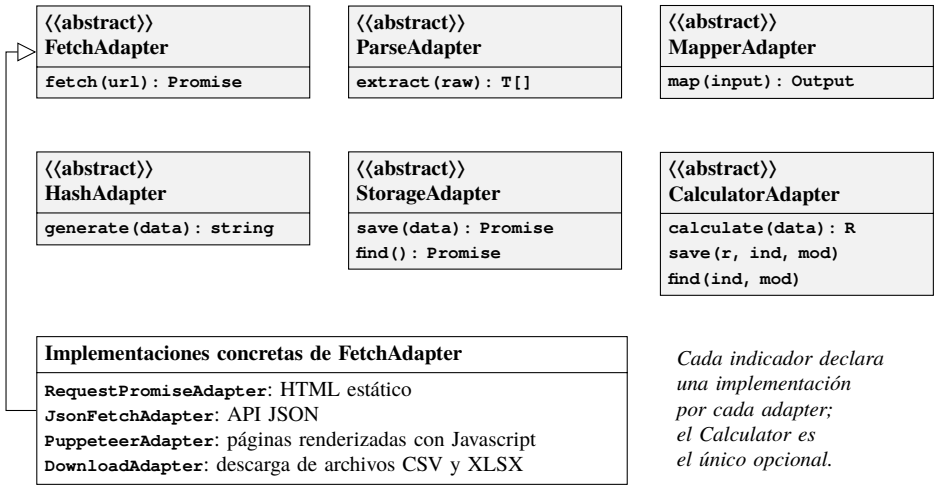


Figura 13: Diagrama de clases de los *adapters* con sus métodos abstractos

6.2.3. Patrón Factory y Singleton

El *ScraperFactory* es una clase *Singleton* que funciona como registro central de módulos. Cada módulo se registra en el *factory* al iniciar la aplicación, y puede ser obtenido por nombre para ejecutar la extracción de un indicador específico. Esto desacopla la lógica de registro de la lógica de ejecución.

De manera similar, el *CronRegistry* es un *Singleton* que registra tareas programadas para cada indicador según su frecuencia configurada, permitiendo la ejecución automática y periódica del proceso de extracción.

6.2.4. Clase abstracta ScrapeBase

Todos los módulos extienden la clase abstracta *ScrapeBase*, la cual implementa el flujo ETL completo. El método `init` orquesta las etapas en el siguiente orden:

1. Se construye la URL reemplazando los parámetros dinámicos (como el año).
2. El *FetchAdapter* obtiene los datos crudos de la fuente.
3. El *ParseAdapter* extrae la información estructurada.
4. El *MapperAdapter* normaliza la estructura de los datos.
5. El *HashAdapter* genera la clave única de cada registro.
6. El *StorageAdapter* almacena los datos en MongoDB, ignorando duplicados.
7. Si existe un *CalculatorAdapter*, se calcula el resultado agregado y se almacena en la colección `indicator-results`, solo si el resultado difiere del último almacenado.

La Figura 14 representa esta secuencia y las decisiones que la condicionan: la interrupción del flujo cuando la extracción no devuelve registros, el descarte de duplicados en el almacenamiento y la escritura condicional del resultado calculado.

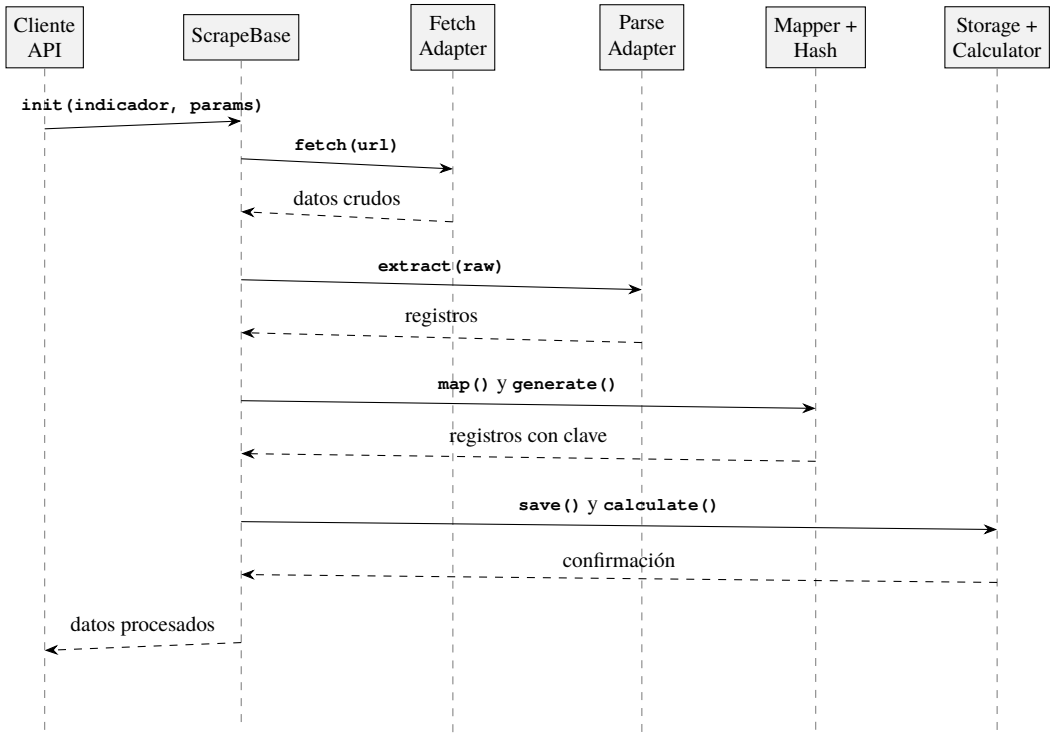


Figura 14: Diagrama de secuencia del flujo ETL completo

6.3. Tipos de conexión implementados

La arquitectura soporta distintos tipos de conexión a fuentes de datos, cada uno implementado como un *FetchAdapter* concreto. Los módulos desarrollados demuestran dos de estos tipos, tal como se resume en la Tabla 9. Sobre la conexión de HTML estático se implementaron además dos estrategias de extracción distintas (tarjetas de simulacros y tablas de reportes comunales), lo que confirma que la variación entre fuentes reside en el *ParseAdapter* y no en la forma de conexión.

Tabla 9: Tipos de conexión implementados en los módulos

FetchAdapter	**Tipo de conexión**	**Módulo**
RequestPromiseAdapter	*Web scraping* de HTML estático	emergencias-desastres, BCN tasa-pobreza-ingresos
JsonFetchAdapter	Consumo de API JSON	BCN organizaciones-comunitarias

Adicionalmente, el *core* incluye dos *adapters* listos para ser utilizados en futuros módulos:

Tabla 10: *Adapters* disponibles para futuros módulos

FetchAdapter	**Tipo de conexión**
PuppeteerAdapter	Páginas renderizadas con Javascript
DownloadAdapter	Descarga de archivos (CSV, XLSX)

Estos *adapters* se encuentran implementados, probados y en funcionamiento; no fueron utilizados en los módulos actuales debido a que las fuentes de datos seleccionadas no lo requerían. Se conservan en el *core* porque el proyecto está pensado para continuar y extenderse: los futuros módulos que requieran descargar archivos CSV o XLSX, o extraer datos de páginas renderizadas con Javascript, podrán declararlos directamente sin desarrollo adicional.

6.4. Módulos implementados

6.4.1. Módulo emergencias-desastres

Este módulo extrae información sobre simulacros de emergencia desde el sitio web de la Unidad de Riesgos y Desastres del MINEDUC. Dado que los *links* originales de esta página ya no se encuentran disponibles, se accede a las versiones archivadas mediante Wayback Machine (Internet Archive, s.f.).

El módulo contiene tres indicadores: simulacros del año 2021, 2022 y 2023. Cada uno utiliza un *ParseAdapter* basado en Cheerio que extrae la fecha, el tipo de evento y la ciudad de cada simulacro a partir de la estructura HTML de la página.

Este módulo además implementa un *CalculatorAdapter* que calcula el total de simulacros y la cantidad por región para cada año, almacenando los resultados en la colección `indicator-results`. Antes de guardar un nuevo resultado, se compara con el último almacenado para evitar registros duplicados cuando los datos de origen no han cambiado. La Figura 15 muestra un registro tal como queda almacenado.

```
{
  "key": "2021-05-27-arica-y-parinacota",
  "date": "2021-05-27T04:00:00.000Z",
  "place": "Sismo Tsunami - Sector Educación",
  "region": "Arica y Parinacota",
  "regionSource": "Arica",
  "indicator": "simulacros-2021",
  "module": "emergencia-desastres"
}
```


Figura 15: Registro almacenado por el módulo emergencias-desastres

El campo **`region`** contiene el nombre canónico de la unidad territorial, mientras que **`regionSource`** conserva el texto tal como lo publica la fuente. Esta distinción responde a que el MINEDUC nombra una misma región de formas distintas según el año; el tratamiento de este problema se detalla en la sección 6.5.

6.4.2. Módulo biblioteca-congreso-nacional

Este módulo contiene dos sub-indicadores que extraen información desde la Biblioteca del Congreso Nacional de Chile (BCN).

El primer sub-indicador, *tasa-pobreza-ingresos*, extrae la tasa de pobreza por ingresos desde la sección de reportes comunales de la BCN. Se utiliza un *ParseAdapter* basado en Cheerio que identifica la tabla de la sección "2.1 Tasa de Pobreza por ingresos" y extrae las filas correspondientes a la comuna, la región y el país, junto con los valores de las encuestas CASEN 2017 y CASEN 2022.

El segundo sub-indicador, *organizaciones-comunitarias*, extrae la cantidad de organizaciones comunitarias por tipo y año desde la API JSON de estadísticas territoriales de la BCN. Se utiliza un *JsonFetchAdapter* que obtiene el JSON directamente y un *ParseAdapter* que extrae el campo **`datosTemaN`** que contiene los datos históricos. En la Figura 16 se muestra un registro de cada sub-indicador.

```
{
  "key": "comuna-de-valdivia-biblioteca-congreso-nacional-...",
  "unidadTerritorial": "Comuna de Valdivia",
  "casen2017": 7.6,
  "casen2022": 3.4,
  "indicator": "valdivia-tasa-pobreza-ingresos",
  "module": "biblioteca-congreso-nacional"
}

{
  "key": "0yhktp6",
  "anio": 2024,
  "nDeJuntasDeVecinos": 175,
  "nDeClubesDeportivos": 180,
  "nDeCentrosUOrganizacionesDelAdultoMayor": 92,
  "nDeCentrosDePadresYApoderados": 35,
  "nCentrosCulturales": 37,
  "nDeCompaniasDeBomberos": 10,
  "nDeUnionesComunales": 2,
  "nDeOrganizacionesComunitariasSumaTotal": 531,
  "indicator": "valdivia-organizaciones-comunitarias",
  "module": "biblioteca-congreso-nacional"
}
```


Figura 16: Registros almacenados por cada sub-indicador del módulo BCN: tasa de pobreza (arriba) y organizaciones comunitarias (abajo)

Los porcentajes de pobreza se almacenan como valores numéricos, convertidos desde el formato decimal chileno con coma que utiliza la fuente. En el caso de las organizaciones comunitarias, el segundo registro se muestra abreviado: cada documento contiene 11 tipos de organización, de los cuales se omiten aquí los que no aportan a la lectura del ejemplo.

6.5. Normalización de unidades territoriales

Durante la validación de los datos extraídos se detectó que la fuente del MINEDUC no aplica ninguna convención al nombrar el territorio donde se realiza cada simulacro. Una misma región aparece escrita de formas distintas según el año de publicación, tal como se detalla en la Tabla 11.

Tabla 11: Grafías distintas de una misma región en la fuente del MINEDUC

Región	Grafías en la fuente	Simulacros
Arica y Parinacota	`Arica` · `Arica y Parinacota`	2
Magallanes	`Magallanes` · `Magallanes y de la Antártica Chilena`	3
La Araucanía	`La Araucanía` · `Lonquimay–Araucanía`	2
O'Higgins	`O'Higgins` · `O´Higgins`	2
Aysén	`Aysén` · `Aysén– Cochrane`	3

El caso de O'Higgins es particularmente ilustrativo, porque la diferencia entre ambas grafías es un único carácter invisible a simple vista: un apóstrofo tipográfico frente a un acento agudo. Sin un tratamiento explícito, los 29 simulacros extraídos producen 21 territorios distintos, en un país que tiene 16 regiones.

Es importante precisar el alcance real del problema. Dentro de un mismo año ninguna región se repite, por lo que los totales por indicador no se ven afectados: los conteos de 9, 11 y 9 simulacros para 2021, 2022 y 2023 son correctos con o sin normalización. La colisión solo se manifiesta al agregar los datos entre años, que es precisamente lo que requiere el indicador *Tsunami Drills* del modelo CORE (definido como la suma de simulacros en un período) y lo que muestran los *dashboards* de Metabase. Se trata, por lo tanto, de un defecto latente antes que de un error en los resultados obtenidos.

La solución adoptada consiste en asignar a cada texto territorial su región canónica mediante coincidencia sobre el nombre sin diacríticos ni mayúsculas, de modo que los prefijos y sufijos que agrega la fuente (`Región de`, `– Provincia de San Antonio`) no impidan la identificación. El registro almacenado conserva ambos valores: el nombre canónico en el campo `region` y el texto original en `regionSource`, lo que permite auditar la extracción contra la fuente sin perder la capacidad de agregar los datos.

Un territorio que no coincida con ninguna de las 16 regiones se conserva sin modificar y se registra en el *log*. Esta decisión es deliberada: descartar el dato ocultaría que la fuente introdujo un territorio nuevo o cambió su forma de nombrarlos, mientras que conservarlo hace visible el cambio sin interrumpir la extracción.

6.6. Manejo de errores

El sistema implementa una jerarquía de errores tipados que permite identificar el origen y el contexto de cada fallo. Los errores se dividen en dos categorías:

- **DomainError**: Errores propios de la lógica del negocio, como un indicador no encontrado (*ParseError*) o un módulo no registrado (*ScraperNotFoundError*).
- **ServiceError**: Errores de servicios externos, como fallos en la conexión HTTP (*FetchError*), en el almacenamiento (*StorageError*) o en la ejecución de tareas programadas (*CronExecutionError*).

Todos los errores heredan de una clase base *BaseError* que incluye un campo de contexto con información adicional. A nivel de la API, un *middleware* centralizado captura estos errores y retorna respuestas JSON con el mensaje correspondiente, evitando que el servidor se detenga ante un fallo.

6.7. Automatización

El *CronRegistry* permite que el sistema ejecute la extracción de indicadores de forma periódica sin intervención manual. Al iniciar la aplicación, el *CronRegistry* recorre todos los módulos registrados y programa una tarea cron para cada indicador que tenga una frecuencia definida.

Las frecuencias se expresan mediante la sintaxis cron estándar y están disponibles como un *enum* predefinido que incluye valores para ejecución diaria, semanal, mensual, anual y única. La ejecución única se utiliza para fuentes históricas que no se actualizan, como es el caso de los simulacros obtenidos desde Wayback Machine.

6.8. API REST

El sistema expone una API REST que permite consultar los módulos, ejecutar la extracción de indicadores y obtener los resultados calculados. Los *endpoints* implementados son los siguientes:

Tabla 12: *Endpoints* de la API REST

Método	Endpoint	Descripción
GET	`/`	Lista todos los módulos con sus indicadores
GET	`/emergencia-desastres`	Lista los indicadores del módulo
GET	`/emergencia-desastres/:indicator`	Ejecuta la extracción del indicador
GET	`/emergencia-desastres/:indicator/result`	Consulta el último resultado calculado
GET	`/biblioteca-congreso-nacional`	Lista los indicadores del módulo
GET	`/biblioteca-congreso-nacional/:indicator`	Ejecuta la extracción del indicador

Cada *endpoint* valida que el indicador solicitado exista antes de ejecutar la extracción. En caso de que no exista, se retorna un error 404 con un mensaje descriptivo.

La API se documenta mediante una colección Bruno (Bruno, s.f.) en formato OpenCollection, versionada junto al código en el directorio `docs/api/`. A diferencia de una documentación redactada aparte, esta colección es ejecutable: cada *endpoint* queda descrito en un archivo YAML que puede ser invocado directamente para verificar la respuesta del sistema. La Figura 17 muestra la definición de uno de ellos.

```yaml
info:
  name: Simulacros 2021
  type: http
  seq: 2

http:
  method: GET
  url: "{{baseUrl}}/emergencia-desastres/simulacros-2021"

settings:
  encodeUrl: true

docs: |-
  Ejecuta el scraping de simulacros del año 2021 desde
  MINEDUC (Wayback Machine).

  Extrae fecha, lugar y región de cada simulacro y los
  almacena en MongoDB.
```

Figura 17: Definición de un *endpoint* en la colección Bruno OpenCollection

6.9. Testing

Se implementaron pruebas automatizadas utilizando el *framework* Jest. Las pruebas se organizan en tres niveles:

En primer lugar, se crearon pruebas unitarias para cada *ParseAdapter*, *MapperAdapter*, *HashAdapter* y *CalculatorAdapter*. Estas pruebas verifican que cada componente funcione de forma aislada con datos de entrada controlados.

En segundo lugar, se crearon pruebas de integración que verifican el flujo ETL completo para cada módulo, desde el *parsing* hasta la generación de la clave única, utilizando datos realistas que simulan la estructura de las fuentes reales.

En tercer lugar, se crearon pruebas de validación que comparan los datos extraídos contra datos de referencia verificados manualmente. Por ejemplo, se verifica que el formato decimal chileno (con coma) sea convertido correctamente al formato numérico estándar (con punto), que las fechas sean parseadas correctamente a partir de los meses en español, y que los nombres de las unidades territoriales se preserven tal como aparecen en la fuente.

El total de pruebas implementadas es de 76, distribuidas en 17 *suites*. Entre ellas se incluye una batería dedicada a la normalización de unidades territoriales descrita en la sección 6.5, que verifica cada una de las grafías observadas en la fuente contra su región canónica.

6.10. Volatilidad de las fuentes web

Uno de los desafíos más significativos que enfrenta un sistema de extracción de datos de la web es la naturaleza cambiante de las fuentes. A diferencia de una base de datos controlada, las páginas web son mantenidas por terceros que pueden modificar su estructura, migrar su contenido o simplemente dejar de estar disponibles sin previo aviso. Un proceso de *scraping* que funciona hoy no necesariamente funcionará mañana.

Durante el desarrollo de este proyecto se experimentó directamente esta problemática. Los *links* originales del sitio web de la Unidad de Riesgos y Desastres del MINEDUC dejaron de estar disponibles antes de completar la implementación del módulo *emergencias-desastres*. Las páginas que contenían los simulacros de los años 2021, 2022 y 2023 fueron removidas del servidor sin que existiera una redirección o aviso previo. Esto implicó que los datos necesarios para el indicador *Tsunami Drills* ya no eran accesibles desde su fuente original.

Para resolver esta situación, se recurrió a Wayback Machine (Internet Archive, s.f.), un servicio de archivo digital de Internet Archive que almacena instantáneas históricas de páginas web. Mediante este servicio se accedió a las versiones archivadas de las páginas

del MINEDUC, lo cual permitió completar la extracción de los datos. Sin embargo, esta solución no es generalizable: Wayback Machine no archiva todas las páginas ni con la misma frecuencia, y su disponibilidad depende de un servicio externo.

Este caso evidencia un conjunto de riesgos inherentes a la extracción de datos desde la web que deben ser considerados en el diseño de sistemas ETL de este tipo:

- **Cambios en la estructura HTML**: Una página puede rediseñar su interfaz sin modificar la URL, invalidando los selectores CSS o XPath que el *ParseAdapter* utiliza para extraer los datos. Este tipo de cambio es silencioso, ya que el sistema no recibe un error explícito sino que extrae datos incorrectos o vacíos.
- **Eliminación o migración de contenido**: Las fuentes gubernamentales y académicas reorganizan sus portales periódicamente. URLs que funcionaban durante meses pueden retornar errores 404 o redirigir a páginas genéricas.
- **Cambios en las APIs**: Las APIs públicas pueden modificar sus parámetros, estructura de respuesta o políticas de acceso (como la implementación de autenticación o límites de tasa).
- **Bloqueo de acceso automatizado**: Algunos sitios implementan mecanismos antibot como CAPTCHAs, límites por IP o detección de *user-agent*, lo cual puede impedir el acceso al sistema de extracción.

La arquitectura modular basada en *adapters* mitiga parcialmente estos riesgos, ya que un cambio en la estructura de una fuente solo requiere modificar el *ParseAdapter* correspondiente sin afectar al resto del sistema. No obstante, la detección del cambio sigue siendo manual: el sistema no distingue entre una extracción que retorna datos vacíos por un error en la fuente y una fuente que simplemente no tiene datos nuevos.

Como trabajo futuro, se propone implementar mecanismos de monitoreo que alerten cuando una extracción retorna resultados inesperados (cero registros, cambios abruptos en la cantidad de datos o en su estructura), permitiendo una respuesta temprana ante cambios en las fuentes.

7. RESULTADOS

En este capítulo se presentan los resultados obtenidos por el sistema desarrollado, tanto en términos de los datos extraídos como de la validación del correcto funcionamiento de la arquitectura.

7.1. Datos extraídos

7.1.1. Tsunami Drills

El módulo *emergencias-desastres* extrajo un total de 29 registros de simulacros distribuidos en tres años:

Tabla 13: Registros extraídos por indicador de simulacros

Indicador	**Registros extraídos**
simulacros-2021	9
simulacros-2022	11
simulacros-2023	9

Cada registro contiene la fecha del simulacro, el tipo de evento (sismo, tsunami, erupción volcánica, remoción en masa, entre otros) y la región donde se realizó. Los datos fueron almacenados en la colección `emergencia-desastres` de MongoDB.

La Figura 18 muestra la respuesta del *endpoint* `GET /emergencia-desastres/simulacros-2021`, que ejecuta la extracción y devuelve los registros ya transformados. En ella se observa el resultado de la normalización territorial descrita en la sección 6.5: el campo `regionSource` conserva el texto original de la fuente (`Arica` en lugar de `Arica y Parinacota`) mientras que `region` contiene el nombre canónico.

Impresión con formato estilístico

```
{
  "data": [
    {
      "date": "2021-05-27T04:00:00.000Z",
      "place": "Sismo Tsunami - Sector Educación",
      "region": "Arica y Parinacota",
      "regionSource": "Arica",
      "indicator": "simulacros-2021",
      "module": "emergencia-desastres",
      "key": "2021-05-27-arica-y-parinacota"
    },
    {
      "date": "2021-06-07T04:00:00.000Z",
      "place": "Remoción en Masa",
      "region": "Tarapacá",
      "regionSource": "Tarapacá",
      "indicator": "simulacros-2021",
      "module": "emergencia-desastres",
      "key": "2021-06-07-tarapaca"
    },
    {
      "date": "2021-06-21T04:00:00.000Z",
      "place": "Sismo Tsumani - Sector Educación",
      "region": "Antofagasta",
      "regionSource": "Antofagasta",
      "indicator": "simulacros-2021",
      "module": "emergencia-desastres",
      "key": "2021-06-21-antofagasta"
    },
    {
      "date": "2021-08-03T04:00:00.000Z",
      "place": "Sismo Tsumani - Borde Costero",
      "region": "O'Higgins",
      "regionSource": "O´Higgins",
      "indicator": "simulacros-2021",
      "module": "emergencia-desastres",
      "key": "2021-08-03-ohiggins"
```

Figura 18: Extracto de los registros de simulacros-2021 mostrando fecha, tipo y región

7.1.2. Resultados calculados de Tsunami Drills

El *CalculatorAdapter* procesó los datos extraídos y almacenó los resultados en la colección **`indicator-results`**:

Tabla 14: Resultados calculados por indicador de simulacros

Indicador	Total de simulacros	Regiones distintas
simulacros-2021	9	9
simulacros-2022	11	11
simulacros-2023	9	9

En los tres años cada región aparece una sola vez, por lo que el total de simulacros coincide con la cantidad de regiones distintas. La Figura 19 muestra el resultado tal como lo entrega el *endpoint* **`GET /emergencia-desastres/simulacros-2021/result`**.

Impresión con formato estilístico

```
{
  "result": {
    "indicator": "simulacros-2021",
    "module": "emergencia-desastres",
    "totalDrills": 9,
    "drillsByRegion": {
      "Arica y Parinacota": 1,
      "Tarapacá": 1,
      "Antofagasta": 1,
      "O'Higgins": 1,
      "Magallanes": 1,
      "Ñuble": 1,
      "La Araucanía": 1,
      "Biobío": 1,
      "Aysén": 1
    }
  }
}
```


Figura 19: Resultado calculado de simulacros-2021 consultado desde la API

Estos resultados permiten a los investigadores consultar de forma directa la cantidad de simulacros realizados por año y su distribución geográfica, sin necesidad de procesar los datos crudos manualmente. Las Figuras 20 y 21 presentan ambas lecturas en Metabase.

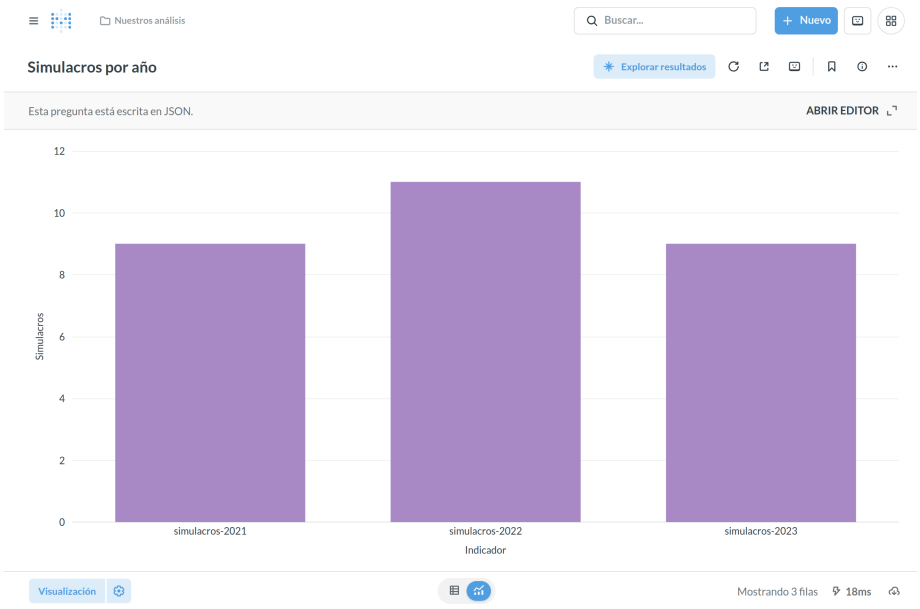


Figura 20: Gráfico de barras de simulacros por año en Metabase

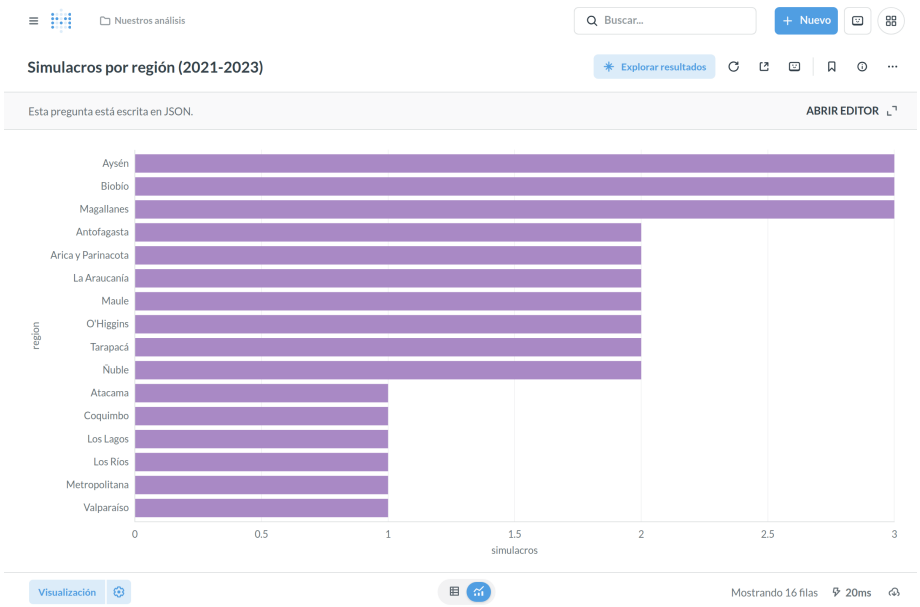


Figura 21: Distribución acumulada de simulacros por región (2021-2023) en Metabase

La agrupación se realiza sobre el campo **`region`** y no sobre **`regionSource`**: de lo contrario una misma región aparecería dividida en varias barras producto de las grafías inconsistentes de la fuente.

7.1.3. Tasa de Pobreza por Ingresos

El sub-indicador *tasa-pobreza-ingresos* extrajo tres registros correspondientes a la comuna de Valdivia, la Región de Los Ríos y el país:

Tabla 15: Datos extraídos de tasa de pobreza por ingresos

Unidad Territorial	CASEN 2017	CASEN 2022
Comuna de Valdivia	7,6 %	3,4 %
Región de Los Ríos	11,8 %	5,9 %
País	8,5 %	6,5 %

Los porcentajes fueron convertidos correctamente del formato decimal chileno (con coma) al formato numérico estándar (con punto), permitiendo su uso directo en cálculos posteriores. La Figura 22 muestra la respuesta de la API y la Figura 23 la comparación entre ambas mediciones en Metabase.

Impresión con formato estilístico

```
{
  "data": [
    {
      "unidadTerritorial": "Comuna de Valdivia",
      "casen2017": 7.6,
      "casen2022": 3.4,
      "indicator": "valdivia-tasa-pobreza-ingresos",
      "module": "biblioteca-congreso-nacional",
      "key": "comuna-de-valdivia-biblioteca-congreso-nacional-valdivia-tasa-pobreza-ingresos"
    },
    {
      "unidadTerritorial": "Región de Los Ríos",
      "casen2017": 11.8,
      "casen2022": 5.9,
      "indicator": "valdivia-tasa-pobreza-ingresos",
      "module": "biblioteca-congreso-nacional",
      "key": "region-de-los-rios-biblioteca-congreso-nacional-valdivia-tasa-pobreza-ingresos"
    },
    {
      "unidadTerritorial": "País",
      "casen2017": 8.5,
      "casen2022": 6.5,
      "indicator": "valdivia-tasa-pobreza-ingresos",
      "module": "biblioteca-congreso-nacional",
      "key": "pais-biblioteca-congreso-nacional-valdivia-tasa-pobreza-ingresos"
    }
  ]
}
```


Figura 22: Respuesta de la API para el indicador de tasa de pobreza por ingresos

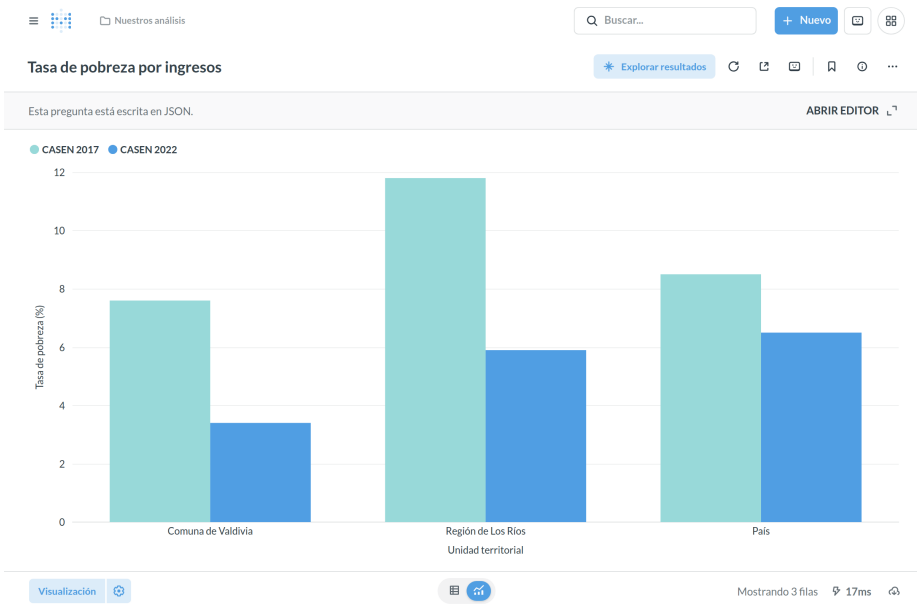


Figura 23: Gráfico comparativo de pobreza CASEN 2017 frente a CASEN 2022 en Metabase

7.1.4. Organizaciones Comunitarias

El sub-indicador *organizaciones-comunitarias* extrajo 17 registros correspondientes a los años 2008 a 2024 para la comuna de Valdivia. Cada registro contiene la cantidad de 11 tipos de organizaciones comunitarias: juntas de vecinos, clubes deportivos, centros de ma-

dres, centros culturales, centros del adulto mayor, uniones comunales, centros de padres y apoderados, compañías de bomberos, entre otros. La serie completa se presenta en la Figura 24.

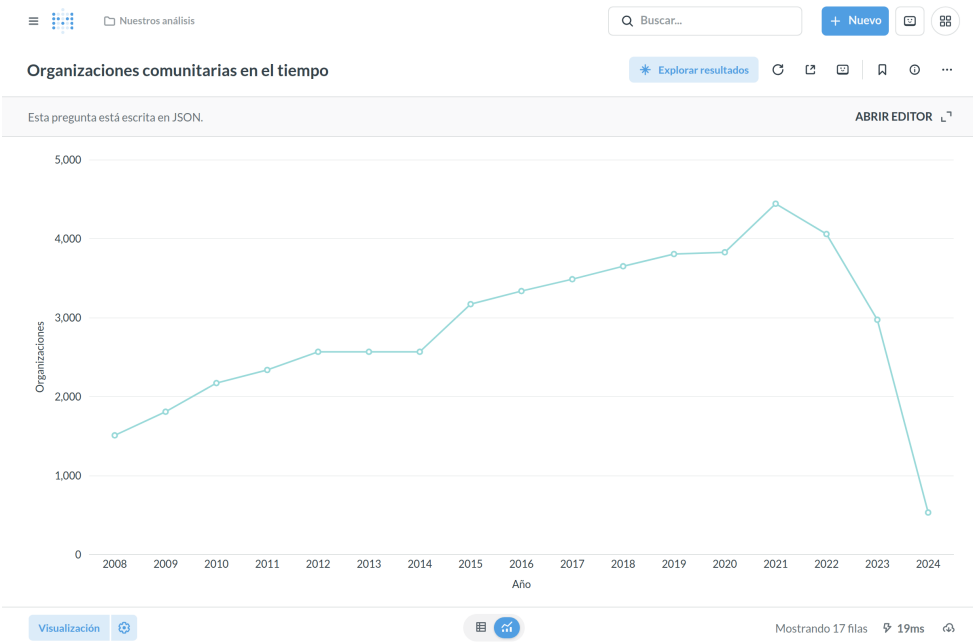


Figura 24: Evolución temporal de las organizaciones comunitarias de Valdivia en Metabase

La construcción de esta serie reveló una particularidad de la fuente que conviene explicitar. La BCN publica, además de la cantidad de cada tipo de organización, un campo con la suma total; sin embargo, para algunos años ese campo viene informado con el carácter – en lugar de un número. El *MapperAdapter* convierte esos valores a cero, lo que hace indistinguible un dato ausente de un cero real: en el año 2017 la suma total quedó almacenada como cero pese a que los tipos individuales suman 3.485 organizaciones. Por esta razón el gráfico recalcula el total a partir de los tipos individuales en lugar de utilizar el campo publicado por la fuente. La caída observada en el año 2024 no responde a este problema, sino a que la fuente aún no publica todos los tipos para ese año.

7.2. La API REST en funcionamiento

La API expone los indicadores mediante los *endpoints* descritos en la Tabla 12. El *endpoint* raíz actúa como punto de entrada y devuelve el catálogo completo de módulos e indicadores registrados en el *ScraperFactory*, junto con la URL de la fuente y la frecuencia de actualización de cada uno, tal como se aprecia en la Figura 25.

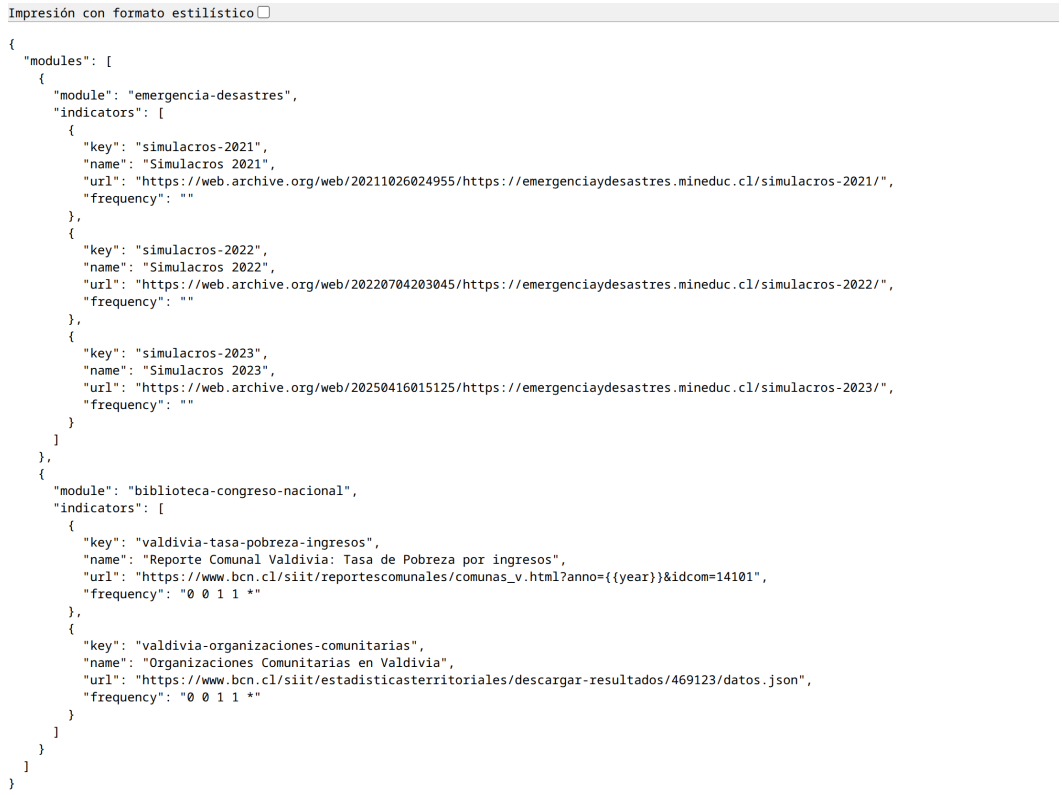


Figura 25: Catálogo de módulos e indicadores devuelto por el *endpoint* raíz de la API

Este catálogo se construye dinámicamente a partir de los módulos registrados, por lo que incorporar un nuevo indicador lo hace visible en la API sin modificar las rutas.

7.3. Visualización en Metabase

Los datos almacenados en MongoDB se visualizan mediante un *dashboard* de Metabase que agrupa los cinco indicadores implementados (Figura 26). Los gráficos consultan directamente las colecciones, de modo que se actualizan de forma automática cada vez que el *CronRegistry* ejecuta una extracción.

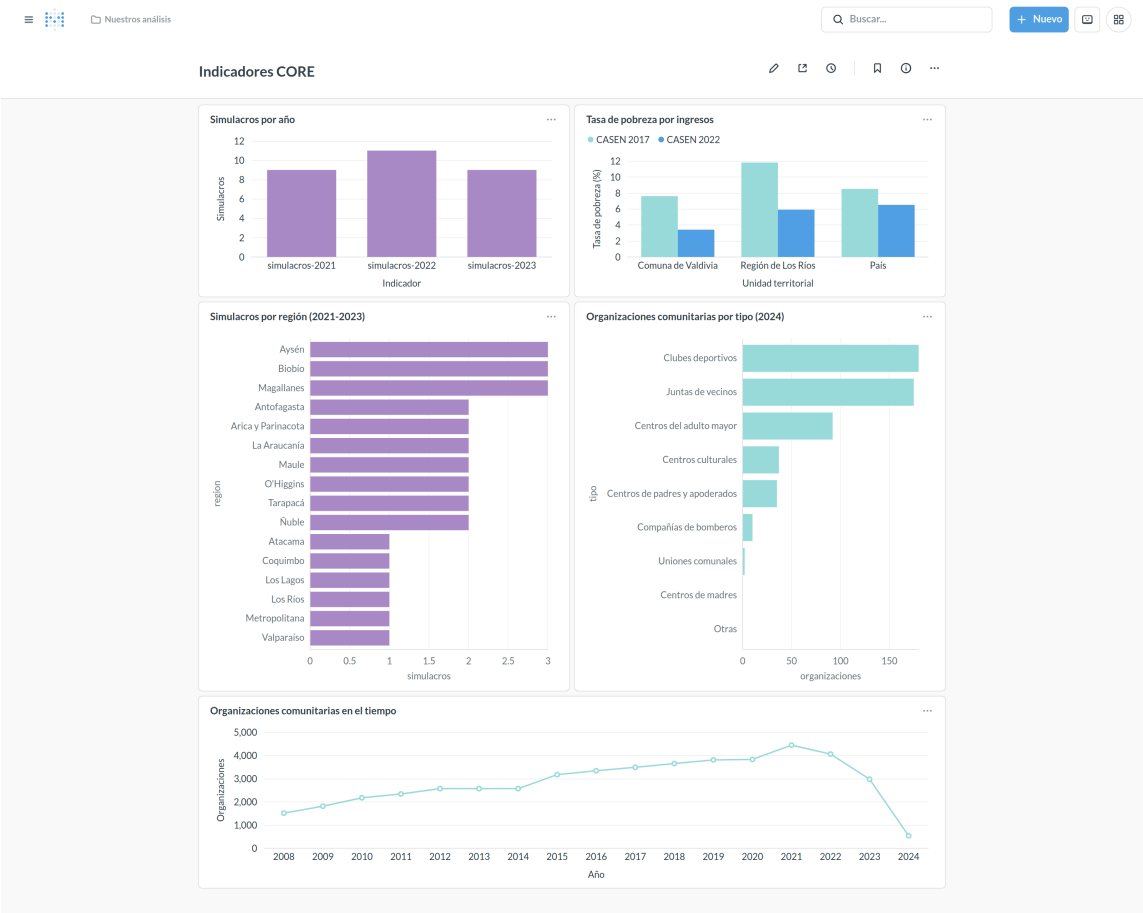


Figura 26: *Dashboard* de Metabase con los indicadores del modelo CORE

Tanto la conexión a MongoDB como las consultas y el *dashboard* se encuentran declarados en un archivo de configuración versionado junto al código, y se aplican automáticamente al levantar el entorno con Docker. De este modo la capa de analítica es reproducible y no depende de una configuración manual de la herramienta.

7.4. Validación de la extracción

Para validar que los datos extraídos por el sistema son correctos, se realizó una comparación con los valores leídos de forma manual en las mismas fuentes. La Tabla 16 resume esta comparación.

Tabla 16: Comparación entre los datos de referencia y los datos extraídos

Indicador	Dato verificado	Fuente	Extraído
simulacros-2021	Simulacros publicados	9	9
simulacros-2022	Simulacros publicados	11	11
simulacros-2023	Simulacros publicados	9	9
tasa-pobreza	Valdivia, CASEN 2017	7,6	7.6
tasa-pobreza	Valdivia, CASEN 2022	3,4	3.4
tasa-pobreza	Los Ríos, CASEN 2017	11,8	11.8
tasa-pobreza	Los Ríos, CASEN 2022	5,9	5.9
tasa-pobreza	País, CASEN 2017	8,5	8.5
tasa-pobreza	País, CASEN 2022	6,5	6.5
organizaciones	Suma total 2023	2.972	2972
organizaciones	Suma total 2024	531	531

En el caso de los simulacros, la verificación consistió en contar las tarjetas publicadas en cada página archivada y contrastarlas con los registros almacenados. Adicionalmente se verificó que las fechas extraídas correspondieran a los meses correctos, considerando que la fuente utiliza abreviaciones en español (Ene, Feb, Mar, entre otras); las pruebas automatizadas validan que cada abreviación sea convertida al índice de mes correcto.

En el caso de la tasa de pobreza se compararon los seis valores publicados en la tabla "2.1 Tasa de Pobreza por ingresos" del reporte comunal de la BCN con los almacenados en la base de datos. La única diferencia es de formato: la fuente utiliza coma decimal y el sistema almacena punto decimal, conversión que también se encuentra cubierta por las pruebas automatizadas.

En el caso de las organizaciones comunitarias se verificó que la suma total publicada por la BCN coincidiera con la suma de los tipos individuales extraídos por el sistema. La coincidencia se cumple en todos los años en que la fuente informa dicho campo; los años en que la fuente lo entrega vacío se discuten en la sección 7.1.4.

7.5. Evitación de duplicados

El sistema fue ejecutado múltiples veces sobre las mismas fuentes de datos para verificar que los mecanismos de evitación de duplicados funcionaran correctamente. El *HashAdapter* genera una clave única para cada registro (basada en la fecha y la región para simulacros, en la unidad territorial para pobreza, y en el año para organizaciones), y los esquemas de MongoDB declaran este campo como único. En cada ejecución posterior a la primera, el sistema detectó los duplicados y los ignoró sin generar errores.

De manera similar, el *CalculatorAdapter* comparó los resultados calculados con el último resultado almacenado y no creó registros nuevos cuando los datos de origen no habían cambiado.

7.6. Rendimiento de la extracción

Se instrumentó el flujo ETL para medir el tiempo de cada etapa por separado. La Tabla 17 presenta los tiempos obtenidos al ejecutar los cinco indicadores sobre colecciones vacías, es decir, en el escenario en que todos los registros deben insertarse.

Tabla 17: Tiempos de ejecución por etapa del flujo ETL, en milisegundos

Indicador	**Reg.**	**Fetch**	**Parse**	**Transf.**	**Save**	**Total**
simulacros-2021	9	881,1	42,2	1,3	13,4	947,6
simulacros-2022	11	172,9	14,4	0,4	4,0	194,3
simulacros-2023	9	172,0	11,4	0,3	3,5	192,7
tasa-pobreza-ingresos	3	333,8	38,5	0,3	3,0	375,5
organizaciones-comunitarias	17	121,6	0,1	0,5	8,3	130,3

La columna *Total* incluye además la etapa de cálculo, presente solo en los indicadores de simulacros. El tiempo mayor del primer indicador se explica por el costo inicial de establecer la conexión HTTP y no por la cantidad de registros: en una segunda ejecución ese mismo indicador bajó a 182,4 ms.

La lectura relevante de estas mediciones no es la magnitud absoluta, sino su distribución: entre el 88 % y el 93 % del tiempo total corresponde a la obtención del documento desde la fuente externa, mientras que las etapas de transformación y persistencia representan en conjunto menos del 10 %. Esto implica que el rendimiento del sistema está determinado por la latencia de las fuentes web y no por su procesamiento interno, por lo que optimizar las etapas propias del ETL tendría un efecto marginal.

En términos agregados, la extracción completa de los cinco indicadores (49 registros) tomó menos de dos segundos. Frente al procedimiento manual que este trabajo busca sistematizar (consistente en navegar cada fuente, copiar los valores y transcribirlos a una planilla), la diferencia no radica únicamente en el tiempo, sino en que el proceso queda documentado, es repetible y no introduce errores de transcripción.

7.7. Ejecución automatizada

Se verificó que el *CronRegistry* programe correctamente las tareas de extracción. Los indicadores de la BCN fueron configurados con frecuencia anual (expresión cron `0 0 1 1 *`) y los simulacros con frecuencia única (sin expresión cron), lo cual es coherente con la naturaleza de cada fuente: los datos de la BCN se actualizan periódicamente, mientras que los datos de Wayback Machine son estáticos.

8. CONCLUSIONES

8.1. Conclusiones generales

El presente proyecto logró desarrollar un sistema de extracción, transformación y carga de datos que sistematiza el proceso de captura de indicadores para el modelo CORE de resiliencia comunitaria. La arquitectura modular basada en *adapters* permite agregar nuevos indicadores y fuentes de datos sin modificar la lógica central del sistema, lo cual constituye el principal aporte técnico del trabajo.

El sistema demostró ser funcional para fuentes de datos de naturaleza distinta (*web scraping* de HTML, extracción de tablas y consumo de APIs JSON), extrayendo y almacenando 49 registros correspondientes a cinco indicadores distribuidos en dos módulos. La validación realizada confirmó que los datos extraídos coinciden con los obtenidos de forma manual, y que los mecanismos de evitación de duplicados operan correctamente ante ejecuciones repetidas.

El ciclo completo que perseguía el proyecto se cierra en los *dashboards* presentados en la sección 7.3: un dato que antes se obtenía navegando manualmente cada fuente y transcribiéndolo a una planilla hoy se extrae, se normaliza, se almacena, se agrega y se grafica sin intervención humana. La distribución de simulacros por región, la comparación de la tasa de pobreza entre dos mediciones CASEN y la serie histórica de organizaciones comunitarias son lecturas que el equipo de investigación puede consultar de forma directa, y que se actualizan solas cada vez que el *CronRegistry* ejecuta una extracción. En ese sentido, el sistema no entrega únicamente datos crudos, sino indicadores en el sentido en que los define el modelo CORE.

8.2. Conclusiones por objetivo

La Tabla 18 resume el grado de cumplimiento de cada objetivo específico y señala dónde se encuentra la evidencia que lo respalda. Las subsecciones siguientes desarrollan cada caso.

Tabla 18: Cumplimiento de los objetivos específicos

OE	Objetivo	Implementado	Evidencia
1	Capturar variables de la dimensión institucional	*Tsunami Drills* (29 registros)	Secciones 7.1.1 y 7.1.2
2	Capturar variables de la dimensión sociocomunitaria	*Population Poverty* y *Civic Organizations* (20 registros)	Secciones 7.1.3 y 7.1.4
3	Poner a disposición los indicadores mediante una API y analíticas	API REST y *dashboards* de Metabase	Secciones 7.2 y 7.3

Los tres objetivos se cumplieron, con una cobertura parcial de indicadores en los objetivos 1 y 2: tres de los seis indicadores analizados quedaron fuera del alcance por las razones que se detallan a continuación. En ningún caso la causa fue una limitación de la arquitectura desarrollada.

8.2.1. Objetivo Específico 1: dimensión institucional

El indicador *Tsunami Drills* fue implementado exitosamente, extrayendo un total de 29 registros de simulacros de emergencia entre los años 2021 y 2023 desde el sitio web del MINEDUC. El *CalculatorAdapter* permitió calcular automáticamente el total de simulacros y su distribución geográfica.

El indicador *Emergency Organizations* (SCEO) no pudo ser implementado debido a la inexistencia de una fuente de datos pública que centralice esta información. Como se documentó en el capítulo de viabilidad de indicadores, estos datos dependen de cada municipalidad y no se encuentran disponibles en una plataforma accesible desde la web. Esta limitación es inherente al dominio de la extracción de datos web: el sistema solo puede capturar información de fuentes que existen y son accesibles. La arquitectura desarrollada permite incorporar este indicador en el futuro si se identifica una fuente viable.

8.2.2. Objetivo Específico 2: dimensión sociocomunitaria

Se implementaron dos indicadores: *Population Poverty* mediante la extracción de la tasa de pobreza por ingresos desde la BCN, y *Civic Organizations* mediante la extracción de organizaciones comunitarias desde la API JSON de la BCN.

Los indicadores *Civic Participation* (SCCP) y *Special Needs Population* (SNP) no fueron implementados dentro del alcance de este proyecto, aunque se identificaron sus fuentes de datos en el capítulo de viabilidad (SERVEL para SCCP, CASEN e INE (Instituto Nacio-

nal de Estadísticas, s.f.-a) para SNP). Dado que estas fuentes ofrecen descargas directas en formato CSV y XLSX, la arquitectura desarrollada ya los soporta mediante el *DownloadAdapter* existente, lo cual facilita su incorporación sin modificaciones al *core* del sistema.

8.2.3. Objetivo Específico 3: API y analíticas

Se logró poner a disposición los indicadores por medio de una API REST y analíticas. La API permite listar los módulos e indicadores disponibles, ejecutar la extracción de datos y consultar los resultados calculados. Los datos almacenados en MongoDB son visualizados mediante *dashboards* en Metabase, los cuales se actualizan automáticamente a medida que se pueblan las colecciones.

Dos decisiones refuerzan el cumplimiento de este objetivo más allá de la mera existencia de los *dashboards*. La primera es que el catálogo de indicadores que expone la API se construye a partir de los módulos registrados y no de una lista escrita a mano, de modo que todo indicador que se incorpore al sistema queda publicado sin tocar las rutas. La segunda es que la configuración completa de la capa de analítica (la conexión a la base de datos, las consultas y el *dashboard*) se encuentra declarada en un archivo versionado junto al código y se aplica sola al levantar el entorno. El resultado es que cualquier integrante del equipo de investigación obtiene el sistema completo, con sus visualizaciones, ejecutando un único comando; la puesta a disposición de los indicadores no queda atada al computador ni a la memoria de quien los configuró.

8.3. Aportes de la arquitectura

Más allá del cumplimiento de los objetivos, el trabajo deja tres aportes que exceden el caso concreto de los indicadores implementados.

El primero es que el *core* del sistema no contiene ninguna referencia al dominio de la resiliencia comunitaria. Las clases *ScrapeBase*, *ScraperFactory*, *IndicatorBuilder* y *CronRegistry*, junto con la jerarquía de *adapters*, operan sobre la noción abstracta de "indicador obtenido de una fuente externa". Todo el conocimiento específico (qué URL consultar, cómo interpretar el HTML, qué campos conforman un registro, cómo se calcula el valor final) reside en los módulos. Esta separación implica que el sistema es reutilizable para cualquier observatorio de indicadores que deba alimentarse de fuentes web heterogéneas, sea en salud, educación, medio ambiente o gestión municipal, sin modificar una línea del *core*. La incorporación de un nuevo indicador se reduce a escribir los *adapters* correspondientes y declararlos mediante el *IndicatorBuilder*.

El segundo aporte es la distinción explícita entre el dato observado y el dato derivado.

El sistema conserva el texto original de la fuente junto al valor normalizado (campos `regionSource` y `region`), y almacena los resultados calculados en una colección separada de los registros crudos. Esta decisión tiene una consecuencia metodológica relevante para un contexto de investigación: permite auditar cualquier valor contra su fuente original y recalcular un indicador con una fórmula distinta sin volver a extraer los datos. En un trabajo científico, donde la trazabilidad del dato es tan importante como el dato mismo, esta propiedad es tanto o más valiosa que la automatización.

El tercer aporte proviene de las mediciones de rendimiento de la sección 7.6. Al concentrarse entre el 88 % y el 93 % del tiempo en la obtención del documento remoto, el procesamiento propio del sistema resulta prácticamente gratuito. Esto significa que el costo marginal de incorporar un indicador adicional es despreciable y que la vía de escalamiento no pasa por optimizar el código, sino por paralelizar y planificar las consultas a las fuentes. La arquitectura, por lo tanto, no solo admite conceptualmente los 21 indicadores que contempla el modelo CORE completo: los admite también en términos de recursos.

8.4. Sobre la calidad de las fuentes de datos públicas

Un resultado transversal del proyecto, no previsto al inicio, es que el principal obstáculo para operacionalizar el modelo CORE en Chile no es técnico sino de disponibilidad y calidad de los datos públicos.

De los seis indicadores de las dimensiones sociocomunitaria e institucional analizados en el capítulo de viabilidad, solo tres cuentan con una fuente pública que permita construirlos de forma automatizada. El indicador SCEO carece por completo de fuente centralizada, y otros dependen de encuestas cuya periodicidad no coincide con la frecuencia con que se querría medir la resiliencia de un territorio.

A esto se suma que las fuentes disponibles presentan problemas de consistencia que solo se hacen visibles al automatizar su lectura. Los 29 simulacros publicados por el MINEDUC dan lugar a 21 nombres de territorio distintos en un país que tiene 16 regiones, al punto de que dos de esas grafías difieren únicamente en un apóstrofo tipográfico frente a un acento agudo. La BCN, por su parte, informa el carácter – donde no dispone de un valor, y publica años con datos parciales sin señalarlo. Ninguno de estos problemas es perceptible para una persona que consulta la página puntualmente; todos ellos comprometen la agregación automática de los datos.

La conclusión que se desprende es que un sistema de extracción de indicadores no puede limitarse a leer y almacenar: debe incorporar una capa de normalización y una política explícita frente a los datos ausentes. El presente trabajo resolvió lo primero mediante el modelo territorial canónico descrito en la sección 6.5, y dejó lo segundo pendiente, como se detalla a continuación.

8.5. Limitaciones

Durante el desarrollo del proyecto se identificaron las siguientes limitaciones:

En primer lugar, la volatilidad de las fuentes web representó el mayor desafío del proyecto. Como se detalló en la sección 6.10, los *links* originales del sitio web del MINEDUC dejaron de estar disponibles durante el transcurso del desarrollo, obligando a recurrir a Wayback Machine para acceder a versiones archivadas. Este no es un caso aislado: la naturaleza cambiante de la web implica que un proceso de *scraping* que funciona hoy puede dejar de funcionar en cualquier momento debido a rediseños de páginas, migraciones de contenido o cambios en las políticas de acceso. Esta realidad debe ser considerada como una restricción fundamental en todo sistema que dependa de la extracción automatizada de datos desde fuentes web no controladas.

En segundo lugar, el indicador SCEO (*Emergency Organizations*) no pudo ser implementado debido a la ausencia de una fuente de datos pública centralizada. La información sobre voluntarios en organizaciones de emergencia depende de cada municipalidad y no se encuentra disponible de manera uniforme.

En tercer lugar, la plataforma Statista fue descartada como fuente para el indicador de pobreza debido a que requiere un pago o suscripción para acceder a sus datos.

En cuarto lugar, la librería *request-promise* utilizada para las peticiones HTTP se encuentra deprecada. Si bien esto no afecta el funcionamiento actual del sistema, representa una deuda técnica que debe ser abordada en futuras iteraciones.

En quinto lugar, el sistema no distingue un dato ausente de un cero legítimo. Los *MapperAdapters* convierten a cero cualquier valor que la fuente no entregue como número, criterio que resultó insuficiente al encontrarse que la BCN informa el carácter – en los campos que no dispone. La consecuencia concreta se documentó en la sección 7.1.4: el total de organizaciones comunitarias del año 2017 quedó almacenado como cero pese a existir 3.485 organizaciones en ese año. El defecto no compromete los valores extraídos correctamente, pero sí puede distorsionar cualquier agregación que los incluya, por lo que constituye la limitación más relevante desde el punto de vista de la calidad de los datos.

En sexto lugar, la detección de fallos silenciosos sigue siendo manual. El sistema arroja un error cuando una extracción no devuelve registros, pero no es capaz de advertir que una fuente cambió su estructura y ahora entrega datos parciales o erróneos manteniendo la cantidad de registros esperada.

8.6. Trabajo futuro

Con base en las limitaciones identificadas y la extensibilidad de la arquitectura, se proponen las siguientes líneas de trabajo futuro:

- Representar el dato ausente de forma explícita, distinguiéndolo del valor cero, e incorporar en cada registro una marca de completitud que permita a las agregaciones y a los *dashboards* excluir los períodos parcialmente informados por la fuente.
- Implementar los indicadores pendientes (SCCP, SNP) utilizando el *DownloadAdapter* ya disponible en el *core* para la descarga de archivos CSV y XLSX.
- Migrar la librería *request-promise* a *fetch* nativo o *axios* para resolver la deuda técnica asociada a la dependencia deprecada.
- Agregar un mecanismo de *retry* con *backoff* exponencial en los *FetchAdapters* para manejar de forma robusta los fallos temporales en las fuentes externas.
- Incorporar un monitoreo que compare cada extracción con la anterior y alerte ante variaciones anómalas en la cantidad de registros o en la distribución de los valores, de modo de detectar los fallos silenciosos descritos en las limitaciones.
- Explorar un enfoque híbrido que combine la arquitectura modular con modelos de lenguaje (LLM) como mecanismo de *fallback* para el *parsing* adaptativo cuando una fuente cambia su estructura HTML.
- Generalizar los módulos de la BCN para soportar múltiples comunas mediante parámetros dinámicos en la URL, de modo de habilitar la comparación territorial que el modelo CORE requiere.
- Extender el modelo territorial canónico al nivel comunal y publicarlo como componente independiente, dado que el problema de las grafías inconsistentes afecta a cualquier sistema que agregue datos públicos chilenos provenientes de fuentes distintas.
- Aplicar la arquitectura a un dominio ajeno a la resiliencia comunitaria para verificar empíricamente la independencia del *core* respecto del dominio.
- Desarrollar un *frontend* personalizado como alternativa a Metabase para casos donde se requiera mayor control sobre la visualización de los datos.

REFERENCIAS

Abrash Walton, A., Marr, J., Cahillane, M. J., & Bush, K. (2021). Building Community Resilience to Disasters: A Review of Interventions to Improve and Measure Public Health Outcomes in the Northeastern United States. *Sustainability*, *13*(21), 11699. https://doi.org/10.3390/su132111699

Atlassian. (s.f.). *What is Version Control*. Consultado el 1 de junio de 2022, desde https://www.atlassian.com/es/git/tutorials/what-is-version-control

Automattic. (s.f.). *Mongoose — Elegant MongoDB Object Modeling for Node.js*. Consultado el 9 de agosto de 2026, desde https://mongoosejs.com/docs/

Biblioteca del Congreso Nacional de Chile. (s.f.-a). *Reportes Comunales — Valdivia*. Consultado el 4 de julio de 2022, desde https://www.bcn.cl/siit/reportescomunales/comunas_v.html?anno=2020&idcom=14101

Biblioteca del Congreso Nacional de Chile. (s.f.-b). *SIIT Estadísticas Territoriales — Organizaciones Comunitarias*. Consultado el 26 de agosto de 2022, desde https://www.bcn.cl/siit/estadisticasterritoriales/tema?id=137

Biblioteca del Congreso Nacional de Chile. (s.f.-c). *SIIT Estadísticas Territoriales — Prevalencia de Personas con Discapacidad*. Consultado el 2 de septiembre de 2022, desde https://www.bcn.cl/siit/estadisticasterritoriales/tema?id=196

Bruno. (s.f.). *Bruno — Opensource IDE for Exploring and Testing APIs*. Consultado el 9 de agosto de 2026, desde https://docs.usebruno.com/

Cheerio. (s.f.). *Cheerio — The Fast, Flexible, and Elegant Library for Parsing and Manipulating HTML and XML*. Consultado el 9 de agosto de 2026, desde https://cheerio.js.org/docs/intro

Cutter, S. L., Ash, K. D., & Emrich, C. T. (2014). The Geographies of Community Disaster Resilience. *Global Environmental Change*, *29*, 65-77. https://doi.org/10.1016/j.gloenvcha.2014.08.005

Docker Inc. (s.f.). *Docker Documentation*. Consultado el 9 de agosto de 2025, desde https://docs.docker.com/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Git. (s.f.). *Git — Distributed Version Control System*. Consultado el 9 de agosto de 2026, desde https://git-scm.com/doc

Glez-Peña, D., Lourenço, A., López-Fernández, H., Reboiro-Jato, M., & Fdez-Riverola, F. (2014). Web Scraping Technologies in an API World. *Briefings in Bioinformatics*, *15*(5), 788-797. https://doi.org/10.1093/bib/bbt026

Google. (s.f.). *Puppeteer — JavaScript API for Chrome and Firefox*. Consultado el 9 de agosto de 2026, desde https://pptr.dev/

Heinzlef, C., Becue, V., & Serre, D. (2020). A Spatial Decision Support System for Enhancing Resilience to Floods: Bridging Resilience Modelling and Geovisualization

Techniques. *Natural Hazards and Earth System Sciences*, *20*, 1049-1068. https://doi.org/10.5194/nhess-20-1049-2020

Instituto Nacional de Estadísticas. (s.f.-a). *Censo de Población y Vivienda 2017*. Consultado el 26 de agosto de 2022, desde https://www.ine.cl/estadisticas/sociales/censos-de-poblacion-y-vivienda/censo-de-poblacion-y-vivienda

Instituto Nacional de Estadísticas. (s.f.-b). *Discapacidad*. Consultado el 2 de septiembre de 2022, desde https://www.ine.cl/estadisticas/sociales/condiciones-de-vida-y-cultura/discapacidad

Internet Archive. (s.f.). *Wayback Machine*. Consultado el 9 de agosto de 2026, desde https://web.archive.org/

Jest. (s.f.). *Jest — Delightful JavaScript Testing*. Consultado el 9 de agosto de 2026, desde https://jestjs.io/docs/getting-started

Kamara, J. K., Agho, K., & Renzaho, A. M. N. (2019). Understanding Disaster Resilience in Communities Affected by Recurrent Drought in Lesotho and Swaziland — A Qualitative Study. *PLoS ONE*, *14*(3), e0212994. https://doi.org/10.1371/journal.pone.0212994

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley.

Maguire, B., & Cartwright, S. (2008). *Assessing a Community's Capacity to Manage Change: A Resilience Approach to Social Assessment*. Bureau of Rural Sciences. Canberra.

McDonnell, C. (s.f.). *Zod — TypeScript-First Schema Validation with Static Type Inference*. Consultado el 9 de agosto de 2026, desde https://zod.dev/

Memoria Chilena, Biblioteca Nacional de Chile. (s.f.). *Terremoto del 27 de febrero de 2010, costa sur del Maule*. Consultado el 10 de agosto de 2026, desde https://www.memoriachilena.gob.cl/602/w3-article-93853.html

Metabase. (s.f.). *Metabase — Business Intelligence, Dashboards and Data Visualization*. Consultado el 9 de agosto de 2025, desde https://www.metabase.com/docs/latest/

Ministerio de Desarrollo Social. (s.f.). *DataSocial — Catálogo de Dimensiones*. Consultado el 26 de agosto de 2022, desde https://datasocial.ministeriodesarrollosocial.gob.cl/portalDataSocial/catalogoDimension/47

Moghadas, M., Asadzadeh, A., Vafeidis, A., Fekete, A., & Kötter, T. (2019). A Multi-Criteria Approach for Assessing Urban Flood Resilience in Tehran, Iran. *International Journal of Disaster Risk Reduction*, *35*, 101069. https://doi.org/10.1016/j.ijdrr.2019.101069

MongoDB Inc. (s.f.). *MongoDB Manual*. Consultado el 9 de agosto de 2025, desde https://www.mongodb.com/docs/manual/

Muhamad, N., Arshad, S. H. M., & Pereira, J. J. (2021). Exposure Elements in Disaster Databases and Availability for Local Scale Application: Case Study of Kuala Lumpur, Malaysia. *Frontiers in Earth Science*, *9*, 616246. https://doi.org/10.3389/feart.2021.616246

Observatorio Social. (s.f.). *Encuesta CASEN 2017*. Consultado el 26 de agosto de 2022, desde http://observatorio.ministeriodesarrollosocial.gob.cl/encuesta-casen-2017

ONEMI. (s.f.). *Chile Preparado — Simulacros*. Consultado el 2 de septiembre de 2022, desde http://chile-preparado.onemi.gov.cl/simulacros/

OpenJS Foundation. (s.f.-a). *Express — Fast, Unopinionated, Minimalist Web Framework for Node.js*. Consultado el 9 de agosto de 2026, desde https://expressjs.com/

OpenJS Foundation. (s.f.-b). *Node.js — Run JavaScript Everywhere*. Consultado el 9 de agosto de 2025, desde https://nodejs.org/en/about

Pino. (s.f.). *Pino — Very Low Overhead Node.js Logger*. Consultado el 9 de agosto de 2026, desde https://getpino.io/

Request. (s.f.). *Request — Deprecated: Simplified HTTP Client* [Proyecto en estado *deprecated* desde febrero de 2020]. Consultado el 9 de agosto de 2025, desde https://github.com/request/request

Rumson, A. G., Payo Garcia, A., & Hallett, S. H. (2020). The Role of Data Within Coastal Resilience Assessments: An East Anglia, UK, Case Study. *Ocean & Coastal Management*, *185*, 105004.

Servicio Electoral de Chile. (s.f.). *Estadísticas del Registro Electoral*. Consultado el 26 de agosto de 2022, desde https://www.servel.cl/estadisticas-2/

Soares, L. M. (s.f.). *node-cron — A Simple Cron-Like Job Scheduler for Node.js*. Consultado el 9 de agosto de 2026, desde https://github.com/node-cron/node-cron

Stack Overflow. (2021). *Stack Overflow Developer Survey 2021*. Consultado el 31 de mayo de 2022, desde https://insights.stackoverflow.com/survey/2021

Statista. (s.f.). *Porcentaje de Personas en Situación de Pobreza en Chile de 2003 a 2020*. Consultado el 26 de agosto de 2022, desde https://es.statista.com/estadisticas/1274561/porcentaje-de-la-poblacion-chilena-bajo-la-linea-de-pobreza/

TypeScript. (s.f.). *TypeScript: JavaScript With Syntax For Types*. Consultado el 31 de mayo de 2022, desde https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html

Unidad de Riesgos y Desastres MINEDUC. (2022). *Simulacros 2022*. Consultado el 1 de septiembre de 2022, desde https://emergenciaydesastres.mineduc.cl/simulacros-2022/

Villagra, P., Herrmann, M. G., Quintana, C., & Sepúlveda, R. D. (2017). Community Resilience to Tsunamis Along the Southeastern Pacific: A Multivariate Approach Incorporating Physical, Environmental, and Social Indicators. *Natural Hazards*, *88*(2), 1087-1111. https://doi.org/10.1007/s11069-017-2908-1

Wang, K., Lam, N. S. N., Zou, L., & Mihunov, V. (2021). Twitter Use in Hurricane Isaac and Its Implications for Disaster Resilience. *ISPRS International Journal of Geo-Information*, *10*(3), 116. https://doi.org/10.3390/ijgi10030116

A. INDICADORES DEL MODELO CORE

La Figura 27 reproduce la tabla de indicadores de la dimensión social del modelo CORE que sirvió de referencia para el análisis de viabilidad del Capítulo 3.

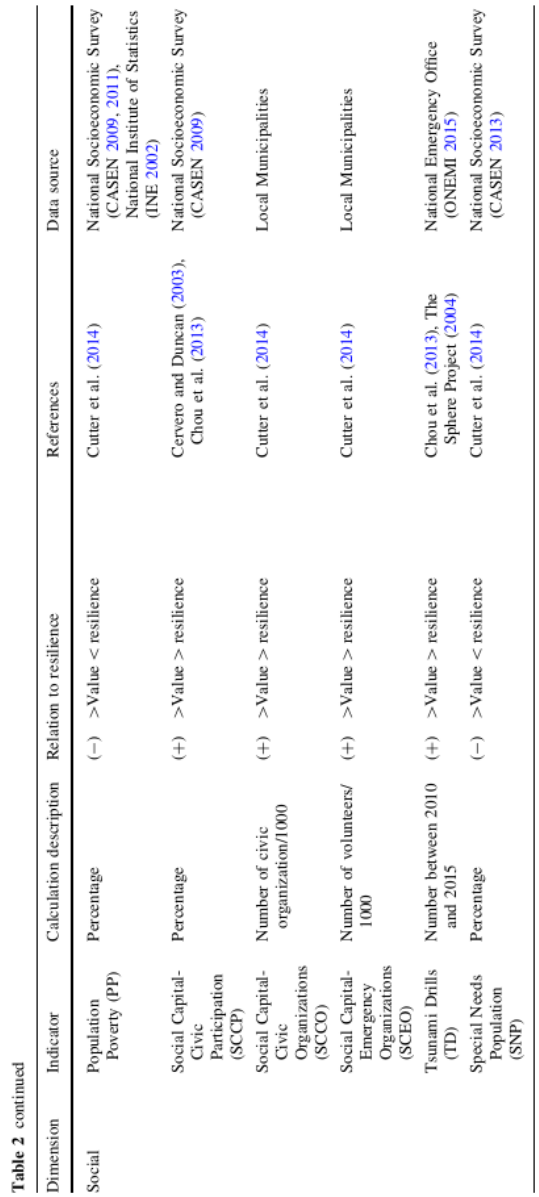


Table 2 continued

Dimension	Indicator	Calculation description	Relation to resilience	References	Data source
Social	Population Poverty (PP)	Percentage	(−) >Value < resilience	Cutter et al. (2014)	National Socioeconomic Survey (CASEN 2009, 2011), National Institute of Statistics (INE 2002)
	Social Capital-Civic Participation (SCCP)	Percentage	(+) >Value > resilience	Cervero and Duncan (2003), Chou et al. (2013)	National Socioeconomic Survey (CASEN 2009)
	Social Capital-Civic Organizations (SCCO)	Number of civic organization/1000	(+) >Value > resilience	Cutter et al. (2014)	Local Municipalities
	Social Capital-Emergency Organizations (SCEO)	Number of volunteers/ 1000	(+) >Value > resilience	Cutter et al. (2014)	Local Municipalities
	Tsunami Drills (TD)	Number between 2010 and 2015	(+) >Value > resilience	Chou et al. (2013), The Sphere Project (2004)	National Emergency Office (ONEMI 2015)
	Special Needs Population (SNP)	Percentage	(−) >Value < resilience	Cutter et al. (2014)	National Socioeconomic Survey (CASEN 2013)

Figura 27: Indicadores de la dimensión social del modelo CORE. Fuente: Villagra et al. (2017)

B. EJEMPLOS DE DATOS EXTRAÍDOS

Este anexo presenta un documento representativo de cada colección de MongoDB, tal como queda almacenado tras la ejecución del flujo ETL. Se omiten los campos que agrega Mongoose de forma automática (`_id`, `__v`, `createdAt` y `updatedAt`).

B.1. Colección emergencia-desastres

Cada documento corresponde a un simulacro. El campo `key` es la clave única generada por el *HashAdapter* a partir de la fecha y la región canónica, y es el mecanismo que impide almacenar un mismo simulacro dos veces.

```
{
  "key": "2021-05-27-arica-y-parinacota",
  "date": "2021-05-27T04:00:00.000Z",
  "place": "Sismo Tsunami - Sector Educación",
  "region": "Arica y Parinacota",
  "regionSource": "Arica",
  "indicator": "simulacros-2021",
  "module": "emergencia-desastres"
}
```

B.2. Colección tasa-pobreza-ingresos

La colección contiene una fila por unidad territorial. Los porcentajes se almacenan como números, convertidos desde el formato decimal chileno que utiliza la fuente.

```
{
  "key": "comuna-de-valdivia-biblioteca-congreso-nacional-
        valdivia-tasa-pobreza-ingresos",
  "unidadTerritorial": "Comuna de Valdivia",
  "casen2017": 7.6,
  "casen2022": 3.4,
  "indicator": "valdivia-tasa-pobreza-ingresos",
  "module": "biblioteca-congreso-nacional"
}
```

B.3. Colección organizaciones-comunitarias

Cada documento agrupa los 11 tipos de organización comunitaria de un año. El ejemplo corresponde al año 2024 e ilustra el problema descrito en la sección 7.1.4: los campos que la fuente no informa quedan almacenados como cero.


```
{
  "key": "0yhktp6",
  "anio": 2024,
  "nDeJuntasDeVecinos": 175,
  "nDeClubesDeportivos": 180,
  "nCentrosCulturales": 37,
  "nDeCentrosUOrganizacionesDelAdultoMayor": 92,
  "nDeCentrosDePadresYApoderados": 35,
  "nDeCompaniasDeBomberos": 10,
  "nDeUnionesComunales": 2,
  "nDeCentrosDeMadres": 0,
  "nDeOtrasOrganizacionesComunitariasFuncionalesOtros": 0,
  "nDeOrganizacionesComunitariasSumaTotal": 531,
  "indicator": "valdivia-organizaciones-comunitarias",
  "module": "biblioteca-congreso-nacional"
}
```


B.4. Colección indicator-results

Esta colección almacena el valor del indicador calculado por el *CalculatorAdapter*, separado de los registros crudos que le dieron origen.


```
{
  "indicator": "simulacros-2022",
  "module": "emergencia-desastres",
  "totalDrills": 11,
  "drillsByRegion": {
    "Arica y Parinacota": 1, "Tarapacá": 1, "Atacama": 1,
    "Valparaíso": 1, "O'Higgins": 1, "Maule": 1, "Ñuble": 1,
    "Biobío": 1, "Los Ríos": 1, "Aysén": 1, "Magallanes": 1
  }
}
```

C. DECLARACIÓN DE UN INDICADOR

Este anexo muestra el código necesario para incorporar un indicador al sistema, con el objeto de dimensionar el esfuerzo real que implica esa tarea. Se utiliza como ejemplo el indicador **`simulacros-2021`** del módulo *emergencias-desastres*. El código fuente completo del sistema se encuentra disponible en el repositorio del proyecto.

C.1. Declaración mediante el IndicatorBuilder

La declaración de un indicador consiste en encadenar los métodos del *IndicatorBuilder* indicando su nombre, la URL de la fuente, la frecuencia de actualización y los *adapters* que resuelven cada etapa del flujo ETL. El *CalculatorAdapter* es opcional y solo se declara en los indicadores que requieren un valor agregado.

```
export const EMERGENCIA_DESASTRES_CONFIG: ModuleConfig = {
  "simulacros-2021": new IndicatorBuilder()
    .setName("Simulacros 2021")
    .setUrl("https://web.archive.org/web/20211026024955/...")
    .setFrequency(FREQUENCIES.once)
    .setFetchAdapter(new RequestPromiseAdapter())
    .setParseAdapter(new DateParserAdapter())
    .setMapperAdapter(new EmergenciaDesastresMapper())
    .setHashAdapter(new EmergenciaDesastresHashAdapter())
    .setStorageAdapter(new EmergenciasDesastresStorageAdapter())
    .setCalculatorAdapter(new TsunamiDrillsCalculatorAdapter())
    .build()
};
```

Los *adapters* de obtención (*RequestPromiseAdapter*) son genéricos y se reutilizan entre indicadores; los restantes son específicos de la fuente.

C.2. Extracción de los datos desde el HTML

El *ParseAdapter* concentra el conocimiento sobre la estructura de la fuente. Es la única pieza que debe modificarse cuando una página cambia su HTML, y su aislamiento permite probarla sin ejecutar el resto del flujo.

```
export class DateParserAdapter implements ParseAdapter {
  extract(data: string) {
    const $ = cheerio.load(data);
    return $(".back-fechas .item .card")
```

```
    .map((_idx, elem) => {
      const dateElem = $(elem).find(".caja-date");
      const date = {
        day: Number($(dateElem).find(".dat_day")
             .text().trim().split("\t")[0]),
        month: $(dateElem).find(".dat_mes").text().trim(),
        year: Number($(dateElem).find(".dat_year").text().trim())
      };
      const placeElem = $(elem).find(".card-body");
      const place = {
        type: $(placeElem).find(".card-title a").text().trim(),
        region: $(placeElem).find(".card-title.pb-3").text().trim()
      };
      return { date, place };
    })
    .get();
  }
}
```